

Controlo Digital da Posição Angular de um Motor CC

Guia da Componente Prática 2 (CP2)

Projeto Integrador em Engenharia Eletrónica Industrial e Computadores 2

Luís Louro, Nuno Ribeiro e Luís Barros

Revisão: 3V6 2026

Licenciatura em Engenharia Eletrónica Industrial e Computadores

Escola de Engenharia da Universidade do Minho

Informação de direitos de autor:

Universidade do Minho

Licença CreativeCommons 3.0 Attribution Share-Alike

1 Introdução

O objetivo principal do guia da Componente Prática 2 (CP2) é a implementação e a validação do controlo digital da posição angular de uma carga rotativa no sistema mecânico. Para a implementação do controlo digital será utilizado o controlador proporcional-integral-derivativo (PID).

A **Figura 1** apresenta o modelo 3D e o protótipo real da maqueta do sistema mecânico, que inclui um motor de corrente contínua (CC), uma carga rotativa e o suporte. O motor (MOT02043, DFRobot¹) já possui um *encoder* (SJ01) e uma caixa redutora de 120:1. O *encoder* gera 16 pulsos em quadratura por volta do motor. Considerando a caixa redutora de 120:1, cada volta do eixo corresponde a 1920 pulsos em quadratura (Sensor A e Sensor B em conjunto, sendo que cada sensor gera 960 pulsos).

Relativamente à carga rotativa, esta possui inércia suficiente para que a constante de tempo de resposta do sistema apresente valores facilmente observáveis. Atualmente, a carga rotativa é apenas um veio ligado ao motor.

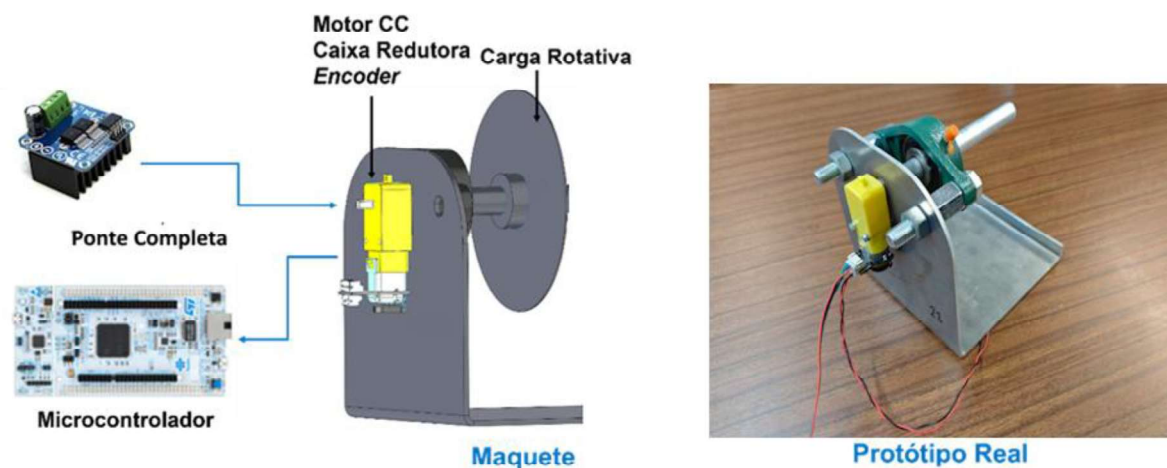


Figura 1 - Representação do sistema mecânico (maquete e protótipo real), do microcontrolador e da ponte completa que serão utilizados para controlar o motor CC do sistema mecânico.

Para implementar o controlo digital da posição angular da carga rotativa, é necessário construir o módulo de sensor e de atuação, seguindo os seguintes dois passos: i) implementação de funções para medir a posição e a velocidade angulares da carga a partir da informação digital monitorizada pelo *encoder* de quadratura; e ii) implementação da função de atuação do motor a partir do microcontrolador (STM32). Para tal será necessária a geração de sinais modulados em largura de pulso, em inglês *Pulse Width Modulation* (PWM), e a sua aplicação a um atuador bipolar (um conversor de eletrónica de potência em ponte completa, vulgarmente designado por *ponte-h*) para acionamento do motor CC. Recomenda-se a utilização de um módulo já com um conversor de eletrónica de potência em ponte completa integrado, como, por exemplo, o driver BTS7960². O terceiro passo, utilizado na secção II deste CP2, consiste na modelização do sistema mecânico e na determinação dos parâmetros do controlador PID, ambos em ambiente

¹ https://wiki.dfrobot.com/Micro_DC_Motor_with_Encoder-SJ01_SKU_FIT0450

² <https://www.handsontec.com/dataspecs/module/BTS7960%20Motor%20Driver.pdf>

de simulação. Por fim, será necessário implementar no microcontrolador a função do controlador PID e validar, em tempo real, o desempenho no protótipo.

O conjunto de conhecimentos de suporte a esta CP inclui matérias de UCs das áreas de Automação e Controlo e de Eletrónica de Potência, que devem ser revistas sempre que necessário.

Ao longo desta CP, todas as simulações serão realizadas no **Matlab®**, enquanto o software para controlar o protótipo real deverá ser implementado no microcontrolador STM32 em linguagem C, utilizando o STM32CubeIDE.

A implementação do programa de controlo a executar nesta CP utilizará uma máquina de estados para gerir o seu funcionamento.

1.1 Objetivos

A CP2 encontra-se estruturada em duas secções: secção I, relativa à implementação da interface com o utilizador e ao desenvolvimento da máquina de estados, do módulo de sensor e de atuação; e secção II, relativa ao sistema de controlo da carga rotativa. Para a secção I, é necessária a criação da máquina de estados e a configuração de todos os periféricos necessários à aquisição e à atuação da carga mecânica. Esta validação deve ser realizada e validada de forma sequencial. Por sua vez, a secção II aborda a modelização do sistema mecânico e a implementação do controlador PID.

Neste trabalho, haverá 2 momentos de **avaliação presencial: o intermédio (5ª aula)**, no qual devem apresentar ao docente o que foi realizado até ao momento (deve estar terminada pelo menos a secção 1), e um **segundo momento na última aula do CP2**, no qual devem realizar as demonstrações de funcionamento da maquete para o objetivo 7. Ao longo do trabalho e para cada objetivo, devem submeter os resultados solicitados na *Blackboard*. Podem ser realizadas questões adicionais ao longo do trabalho.

Tabela 1 - Definição das datas para a apresentação dos objetivos.

Objetivo(s)	Aulas
0	1ª aula
1	2ª aula
2	4ª aula
3	
4	6ª aula
5	
6	7ª aula

O objetivo 0 deve ser submetido antes do início da primeira aula. Os restantes devem ser submetidos no final das aulas indicadas. A não apresentação do objetivo no *Blackboard* na data estipulada acarretará uma **penalização de 30% por aula de atraso**.

2 Sistema de controlo

Um sistema de controlo é uma combinação de subsistemas e processos, com o propósito de obter a saída e o desempenho desejados a partir de uma entrada específica.

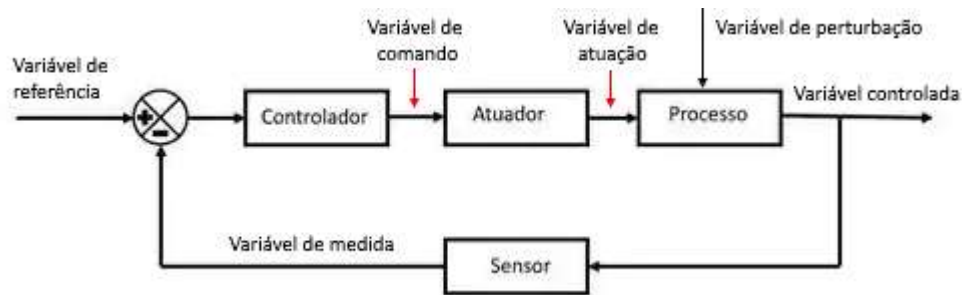


Figura 2 - Esquema de um sistema de controlo em malha fechada.

Um sistema de controlo pode ser realizado de diversas formas, sendo que neste trabalho será explorado um sistema de controlo em malha fechada constituído pelos seguintes subsistemas (**Figura 2**):

- O processo/sistema a ser controlado
- O elemento atuador
- O elemento sensor
- O controlador

Cada um destes subsistemas tem na sua entrada e gera na sua saída um conjunto de variáveis:

- A variável controlada
- A variável de atuação
- A variável de perturbação
- A variável de comando
- A variável medida
- A variável de referência

2.1 Sistema/processo

O sistema a ser controlado é mostrado esquematicamente na **Figura 1**. É um disco rotativo fixado (atualmente, apenas o veio associado ao motor) por meio de um eixo a um suporte mecânico. O objetivo do sistema de controlo será:

- Rodar o disco a partir de uma referência, em um ângulo determinado (possivelmente variando ao longo do tempo).
- Compensar qualquer perturbação que possa tirar o disco da posição de referência.

2.2 Atuador

É composto pelo motor CC com caixa de velocidades (reductor), pela ponte completa e, parcialmente, pelo microcomputador. O eixo do motor CC liga-se ao eixo do disco por meio de um reductor. O eixo de saída da caixa de engrenagens aciona o disco. Desta forma, a velocidade no eixo do disco é reduzida em relação à do eixo do motor, e o binário é aumentado na proporção inversa. A quantidade de binário produzida pelo motor CC pode ser influenciada por meio da alteração da tensão de alimentação. O sistema de controlo ajustará esta tensão para que

o binário produzido seja adequado. A tensão média de alimentação do motor CC será fornecida por um amplificador PWM, que consideraremos com ganho unitário.

2.3 Sensor

O componente primário é um *encoder* incremental acoplado ao motor CC. Este gera duas sequências de impulsos retangulares em quadratura. Se estes forem aplicados às entradas de um microcomputador, este pode calcular a posição e a velocidade do eixo ao i) detetar a direção dos impulsos em quadratura e ii) contar o número de impulsos recebidos num período de tempo.

Com a disposição acima, obtém-se um sensor de posição/velocidade composto pelos elementos do motor CC e, novamente e parcialmente, pelo microcomputador. Os valores que representam a posição e a velocidade angular são calculados no microcomputador.

2.4 Controlador

O controlador será implementado no microcomputador. A sua principal componente é um ISR (interrupção) que implementa a regra de controlo. Este ISR será invocado por um temporizador definido para desencadear interrupções com um período igual ao da amostragem do sistema de controlo. O ISR irá exibir os valores de medição da posição angular e os valores de referência e calculará um valor de alimentação de tensão a ser gerado pelo amplificador PWM. Outra função que o microcomputador deve implementar é a interface com o utilizador, para que este possa definir parâmetros de controlo, receber informações sobre o funcionamento do sistema de controlo, iniciar e parar o controlador, entre outras.

3 Máquina de Estados

Uma máquina de estados é um modelo matemático utilizado para representar sistemas que podem ser descritos por um conjunto finito de estados distintos e que transitem entre eles de acordo com eventos ou condições predefinidas.

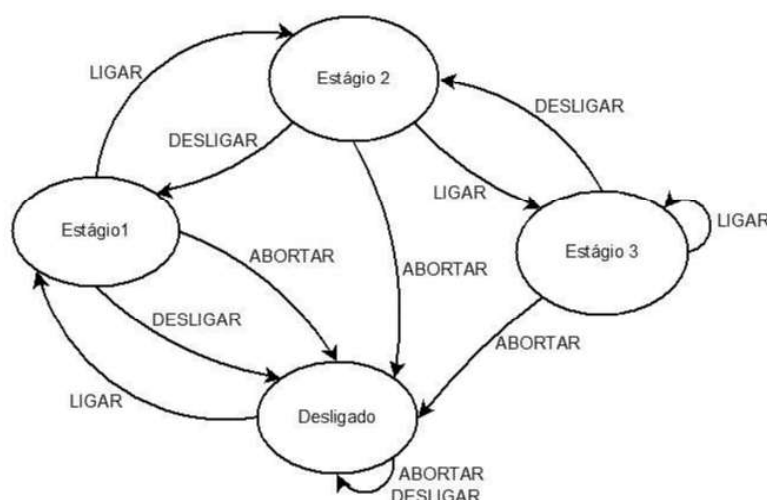


Figura 3 - Exemplo de uma máquina de estados.

Neste projeto, pretende-se criar 4 estados/modos de funcionamento do programa, em que cada estado permite a execução de determinadas funções e limita a execução ou a não execução de comandos. A transição entre estados também será norteadada por um conjunto de regras.

Os 4 estados são:

0. *Modo de reset;*
1. *Modo de configuração;*
2. *Modo de leitura/comando manual;*
3. *Modo automático – controlo PID em malha fechada.*

Estes devem ser indicados no início do funcionamento do programa principal do microcontrolador. Uma descrição do que é realizado em cada estado e do modo de transição entre estados é apresentada seguidamente:

0) *Modo de reset:*

- Desativa os pinos de *enable*, de definição de sentido de rotação e de sinal de PWM;
- Modo de limpeza das configurações iniciais do utilizador (período de amostragem – 10 ms; tipo de leitura - leitura de posição; *reset* de posição, PWM - *duty cycle* 0%, valores dos parâmetros PID – todos a zero);
- Deve ser desativado qualquer periférico em atuação;
- Transição de estado: o estado 0 deve ser transitório, passando para o *estado 1 (modo de configuração)* após o *reset* das variáveis ser concluído.

1) *Modo de configuração:*

- Este modo serve para o utilizador configurar diferentes variáveis necessárias para o controlo:
 - Configuração do período de amostragem (utilizador);
 - Configuração do tipo de leitura a ser realizada (utilizador);
 - Configuração do valor inicial do *duty-cycle* do sinal de PWM (utilizador);
 - Configuração da posição zero do disco - contador de posição inicializado (utilizador);
 - Configuração da posição desejada do disco e dos parâmetros do controlador em malha fechada (utilizador);
 - Configuração dos periféricos necessários (microcontrolador).
- Subentende-se que sempre que este estado é iniciado, deve-se limpar os somatórios de erros existentes no controlador PID;
- Subentende-se que ao entrar neste modo de funcionamento, a variável de *Enable do motor* (EN) deve tomar o valor de 0, e o microcontrolador deve inicializar os restantes periféricos (PWM, GPIO, ...);
- Transição de estado:
 - O *estado 1 (Modo de configuração)* pode ser selecionado a partir de qualquer um dos outros estados;

- A partir do estado 1 (Modo de configuração) é possível seleccionar qualquer outro estado da máquina de estados.

2) Modo de leitura/comando manual:

- Este modo serve para leitura de posição e velocidade angular a partir dos valores do *encoder* (funções a definir no objetivo 2) e para manipulação contínua e direta do sinal de PWM (funções a definir no objetivo 3);
- Neste modo é possível:
 - Solicitar pedidos de amostras (utilizador);
 - Enviar comandos de PWM, seja por comando ou usando o teclado (utilizador);
 - Alterar a posição 0 do motor - contador de posição inicializado (utilizador);
 - Realizar o *Enable do motor* (utilizador);
- Relativamente ao comando do motor via PWM, apenas passa a ser atuado após a ativação do comando *Enable do motor* (EN=1). Um para cada sentido de rotação (para a ponte completa do BTS7960).
- A desativação do *Enable do motor* ou o pedido de paragem de leituras não deve fazer o programa sair do estado 2 (Modo de leitura/comando manual);
- Transição de estado:
 - O estado 2 (Modo de leitura/comando manual) é ativado pelo acionamento do CS=2, apenas a partir do estado 1 (Modo de configuração);
 - O estado 2 (Modo de leitura/comando manual) deve permanecer ativo até o *estado 0 (modo de Reset)* ou o *estado 1 (Modo de configuração)* ser seleccionado. Não é possível passar para o *estado 3 (modo automático)*

3) Modo automático – controlo PID em malha fechada:

- Este modo visa o teste do controlo do disco usando o controlo PID implementado. O programa a realizar para este estado será definido no objetivo 6;
- Após a ativação do *estado 3 (Modo automático)*, o sistema deve ligar o *Enable do motor* (EN=1) e arrancar o teste em malha fechada;
- Este modo, depois de ativo, deverá permanecer em funcionamento independentemente de qualquer perturbação externa causada à carga mecânica;
- Caso o utilizador desative o *Enable do motor* (EN = 0), o motor deve parar e o sistema volta ao *estado 1 (modo de configuração)*.
- Ao sair deste modo, o sinal de *Enable do motor* deve ser desativado;
- Transição de estado:
 - O *estado 3 (Modo automático)* é ativado pelo acionamento do CS=3, a partir do *estado 1 (Modo de configuração)*.
 - O *estado 3 (Modo automático)* deve permanecer ativo até ao utilizador voltar a seleccionar o estado 1 (Modo de configuração) ou o *estado 0 (Modo de Reset)*. Se realizar o *disable* (EN=0) do motor deve passar para o *estado 1 (Modo de configuração)*

Objetivo 0

Neste objetivo inicial deve apresentar:

- **Diagrama de blocos** que identifique os elementos constituintes do projeto (motor, ponte completa, STM32 e *Encoder*), identificando todas as ligações que têm de ser realizadas entre cada bloco (*inputs e outputs*), e indicando os valores de tensão máxima aplicados em cada uma das ligações (ex.: para a STM32 identificar os níveis de tensão de entrada e saída);
- **Esquemático do sistema de controlo:** deve ser apresentado o esquema do sistema de controlo da posição angular do sistema mecânico, uma lista das variáveis de controlo (variável controlada, de atuação, de perturbação, de comando e medida) e deve ser indicado como são constituídos os 4 subsistemas do sistema de controlo (sistema, atuador, controlador e sensor);
- **Diagrama da máquina de estados:** indicar os diferentes estados e suas funcionalidades, as transições entre eles e os comandos que podem ser executados em cada um dos estados;

NOTA: Atenção, um diagrama de blocos não é a mesma coisa que um esquemático do sistema de controlo. O documento relativo a este documento deve ser submetido no diário da *blackboard* antes do início da primeira aula.

4 Secção I – Implementação da máquina de estados, interface e módulo sensor e de atuação

4.1 Programação do microcontrolador com um controlador digital

A implementação geral de um algoritmo digital utiliza uma estrutura concorrente, como ilustrada na **Figura 4**. A estrutura é concorrente porque o *software* é distribuído em módulos que executam em paralelo e comunicam entre si por meio de posições de memória. Esta organização é adequada a sistemas com restrições de tempo real, em que a amostragem da variável deve ser periódica e o cálculo da variável de comando ou de outras grandezas de processamento necessárias ao sistema deve ser concluído antes do próximo instante de amostragem. No caso de algoritmos de controlo digital, o cálculo do valor da variável de comando deve ser realizado num intervalo de tempo inferior ao período de amostragem de forma a permitir acomodar a restante computação necessária.

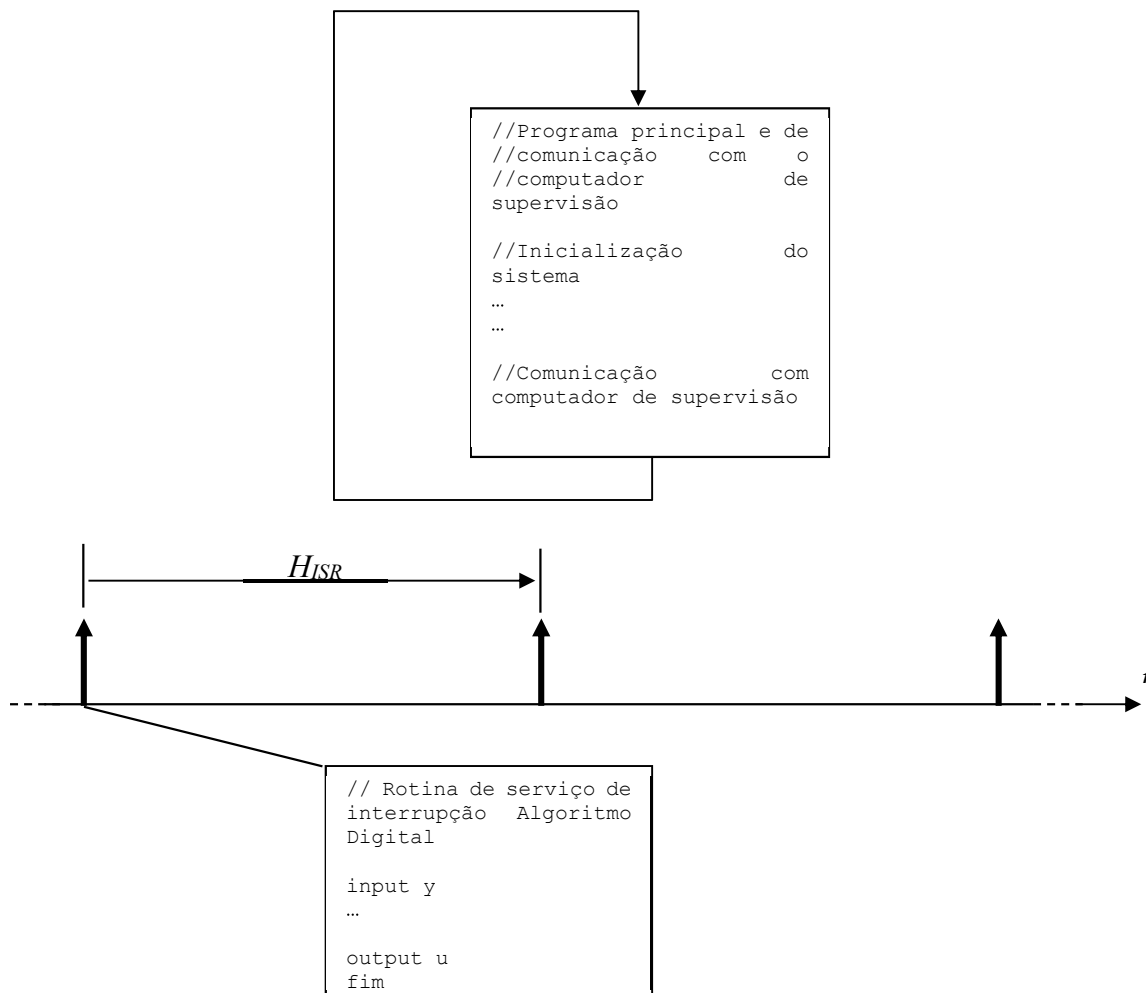


Figura 4 - Exemplo de estrutura do software para a implementação de um algoritmo digital em tempo real. O programa principal realiza a inicialização do sistema e a comunicação com o computador de supervisão. Em particular, é o responsável pela inicialização do temporizador que gera as interrupções para a respetiva rotina de serviço (ISR). Se a variável de saída corresponder, por exemplo, a uma fração de modulação PWM, o respetivo gerador deve ser atualizado.

Primeiro, deve estabelecer-se um programa principal e um programa de comunicação com o

computador de supervisão ou um programa de interface com o utilizador. Este programa realiza a programação dos temporizadores e das interrupções.

A comunicação com o computador local permitirá ao utilizador definir os parâmetros do algoritmo, como certas variáveis, como a variável de referência, e os modos de funcionamento. O programa principal comunica-se com a rotina de serviço de interrupção por meio de variáveis em memória a que ambos acedem.

Segundo, deve-se estabelecer um mecanismo que realize a amostragem com uma frequência determinada da variável a ser controlada. Este mecanismo é obtido por meio da programação de um temporizador que gera interrupções. A rotina de serviço de interrupção (ISR), ativada periodicamente por um temporizador, executa o algoritmo definido para a aplicação, que pode corresponder a um controlador (por exemplo, PID), a um algoritmo de estimação de velocidade ou posição, a um cálculo de variáveis derivadas ou a outro processamento digital relevante. No caso do algoritmo PID, deve calcular o valor do erro e , a partir dele e dos parâmetros do controlador, determinar o valor do comando, emitindo o *output* com esse valor. O algoritmo escreve o valor do comando em memória, e outro processo, por exemplo, um modulador PWM digital, utiliza esse valor para estabelecer o *duty cycle* (e a polaridade) da tensão aplicada.

Assim, em cada interrupção, tipicamente são realizadas três operações: leitura da variável medida, execução do processamento ou do algoritmo implementado e escrita do valor resultante.

Sendo os instantes de tempo de amostragem estabelecidos por um relógio que gera interrupções, a cada interrupção, as três operações indicadas acima devem ser realizadas pela rotina de interrupção. Note-se que a soma dos tempos de execução da rotina de interrupção do PID com os tempos de execução de outras rotinas de interrupção (PWM, elemento sensor) deve ser inferior ao período de amostragem, de modo a permitir a execução do programa principal.

Objetivo 1- Implementação da máquina de estados, interface e comandos a utilizar neste guia

Preparação de aula: devem ter preparado o algoritmo (um esboço) da máquina de estados a implementar, considerando a máquina de estados delineada no objetivo 0. Devem também ter analisado os comandos a serem implementados, a gramática por trás deles e os estados em que podem ser utilizados.

Neste objetivo, os alunos devem implementar a máquina de estados delineada no objetivo 0, seguindo as condições de transição e as regras de comando aplicáveis a cada estado. O interpretador dos comandos deve já estar implementado neste objetivo, incluindo a possibilidade de execução em cada estado, ficando a funcionalidade de cada uma das funções para ser implementada nos restantes objetivos. O sistema a implementar necessita de uma interface com o utilizador, do tipo linha de comandos, entre o microcontrolador e o computador local, por meio de comunicação série, via norma USB ou RS-232. Como programa de interface no PC podem utilizar o *Terminal* ou similares. Esta interface deve permitir a indicação do **estado de funcionamento** do microcontrolador, bem como a **leitura e a escrita, em tempo real, dos valores das variáveis e dos parâmetros do controlador PID**. Para a implementação da interface, podem reaproveitar as implementações de guias anteriores, considerando a gramática de interface apresentada de seguida:

Gramática geral de interface

COMANDO = <char>⁺ <CR>|<char>⁺ <control char>
 <char> = {'a'..'z','A'..'Z'} U {'0'..'9'} U {'+', '-'} U {β, <bckspc>}
 <control char> = {<esc>,'\$'}
 'β' = 20h
 <esc> = 1Bh
 <bckspc> = 08h
 <uint>, <16 bit value> = {"0000".."FFFF" | "0".."65536"}
 <dig> = {"0".."9"}
 <sign> = {'|'+','|'-'}
 <signval> = <sign><uint>⁺
 <float> = <dig> "." <dig>

Implementação da máquina de estados:

Implemente a máquina de estados com base naquela delineada no objetivo 0. A implementação computacional destes estados de funcionamento deve ser realizada no programa principal por meio de um *switch case*, que, por sua vez, permite a ativação de funções específicas para cada estado.

Comandos a configurar:

Em seguida, apresenta-se o conjunto de comandos a ser executado durante a CP. O interpretador de comandos deve ser implementado neste objetivo, com a implementação da maior parte dos comandos a ser realizada nos restantes objetivos. Os comandos devem aceitar apenas os valores indicados no próprio comando; caso contrário, deve ser devolvida uma mensagem de erro.

- a) **CS – Control System:** este comando permite definir o estado de funcionamento da máquina de estados. Este comando deve ficar já funcional, garantindo a alteração de estado:

Control system: <char> += "CS β <dig>+"

- i. <dig> = 0 - Para Modo de reset;
- ii. <dig> = 1 - Para Modo de configuração;
- iii. <dig> = 2 - Para Modo de leitura/comando manual;
- iv. <dig> = 3 - Para Modo automático;

NOTA: Este comando pode ser utilizado em todos os estados.

- b) **EN – Enable dos motores:** este comando ativa a atuação dos motores; ou seja, aciona o pino de ENABLE, que, por sua vez, ativa o funcionamento da ponte completa. A implementação do comando pode ser concluída posteriormente no objetivo 3.

Enable dos motores: <char> += "EN β <dig>+"

- i. <dig> = 0 - Desativa o controlo dos motores;
- ii. <dig> = 1 - Ativa o controlo e o pino de ENABLE.

NOTA: Este comando pode ser utilizado nos estados 2) *modo de leitura/comando manual* e 3) *modo automático*. Nos restantes estados, o *Enable dos motores* deve estar a 0 e não pode ser alterado.

- c) **HW** – *Período de amostragem*: Este comando permite configurar o período de amostragem, utilizado tanto para a amostragem quanto para o cálculo da posição e da velocidade, em *modo de leitura/comando manual* (2) e em *modo automático* (3). Deve ser considerada uma base de 1 ms. O interpretador de comando deve ser implementado neste objetivo, mas a implementação da função pode ser realizada durante a execução do objetivo 2. Para a implementação do cálculo manual da configuração da interrupção do *timer* (ver objetivo 2).

Período de amostragem: $\langle \text{char} \rangle += \text{"HW } \beta \langle \text{uint} \rangle \text{"}$

- i. $\langle \text{uint} \rangle$ - Deve apresentar um valor compreendido entre 1 e 1000, apresentando assim um período de amostragem de 1 ms a 1 s, respetivamente;

NOTA: Para configurar um período de amostragem de 20 ms, o comando deve ser "HW 20". Este comando pode ser usado apenas no estado 1) Modo de configuração. Por defeito, o valor de amostragem deve estar configurado para 10ms. Valores acima de 1000 devem retornar uma mensagem de erro. Por defeito, o valor do período de amostragem deve ser definido a 100ms.

- d) **RT** – *Configuração do tipo de leitura*: este comando permite definir o tipo de leitura a ser utilizada. O interpretador do comando já deve estar implementado, sendo a função implementada no objetivo 2.

Reading Type: $\langle \text{char} \rangle += \text{"RT } \beta \langle \text{dig} \rangle \text{"}$

- i. $\langle \text{dig} \rangle = 0$ - Definir para leitura de posição;
 ii. $\langle \text{dig} \rangle = 1$ - Definir para leitura de velocidade;
 iii. $\langle \text{dig} \rangle = 2$ - Definir para leitura de posição e velocidade;

NOTA: Para imprimir o valor da velocidade e da posição ao mesmo tempo, o comando deve ser "RT 2". Este comando pode ser utilizado apenas no estado 1) Modo de configuração. Por defeito, o valor da leitura está definido para 0 (leitura de posição).

- e) **R** – *Leitura*: Este comando permite iniciar ou interromper a leitura de amostras. Permite definir o tipo de leitura (contínua ou limitada) e o número de amostras desejado. A implementação da funcionalidade deste comando pode ser realizada no objetivo 2.

Reading: $\langle \text{char} \rangle += \text{"R } \beta \langle \text{dig} \rangle \beta \langle \text{uint} \rangle \text{"}$

- i. $\langle \text{dig} \rangle = 0$ - Termina a amostragem, seja do modo contínuo ou do modo limitado.
 ii. $\langle \text{dig} \rangle = 1$ - Pedido de leituras contínuas, para um período de amostragem definido pelo período de amostragem (HW).
 iii. $\langle \text{dig} \rangle = 2 \ \& \ \langle \text{uint} \rangle \geq 1$ - Com $\text{dig} = 2$, é acionado o pedido de leituras limitadas, com a quantidade de amostras indicada em $\langle \text{uint} \rangle$ (número máximo de amostras de 1000 para a amostragem limitada), para um período de amostragem definido por HW.

NOTA: Para a configuração de amostragem contínua, o comando deve ser "R 1". Para o modo limitado, de 250 amostras, fazer "R 2 250". Para terminar a amostragem "R 0". Este comando pode ser utilizado apenas no estado 2) *Modo de leitura/comando manual*.

- f) **PWM – Tensão Normalizada:** este comando permite definir o valor da tensão normalizada (*duty-cycle*) a aplicar. Mais uma vez, a funcionalidade deste comando deve ser implementada no objetivo 3. Para a implementação do cálculo manual do *duty cycle* do PWM, ver o objetivo 3.

Tensão Normalizada: $\langle char \rangle += \text{“PWM } \beta \langle signval \rangle +”$

- i. $\langle signval \rangle$ - Valor compreendido entre +100 (máx. FW) e –100 (máx. RV), com incrementos de 1. O valor absoluto indica a percentagem de PWM (ou de *duty cycle*) e o sinal indica o sentido da tensão aplicada à carga (pino FW/RV). A tensão média aplicada ao motor igualará a tensão da fonte vezes a Tensão Normalizada a dividir por 100 ($V_{cc} * \langle signval \rangle / 100$);
- ii. $\langle signval \rangle$ - Deve ler-se como $\langle sign \rangle \langle uint \rangle^+$.

NOTA: Para um *duty-cycle* de 50% no sentido *reverse*, o comando deve ser “PWM -50”. Para um *duty-cycle* de 25% no sentido *forward*, o comando deve ser “PWM +25”. Este comando pode ser usado nos estados 1) *Modo de configuração* e 2) *Modo de leitura/comando*. Por defeito, o *duty cycle* deve ser definido para 0.

- g) **RESPOS – Reinicialização da posição 0 do disco:** Este comando permite definir a posição atual do disco como a posição zero, limpando o contador de posições do *encoder*. A funcionalidade deste comando deve estar implementada para o objetivo 2, sendo o comando útil tanto para o estado 2) *modo de leitura/comando do motor* e 3) *modo automático*.

Reinicializa a posição zero do disco: $\langle char \rangle += \text{“RESPOS ”}$

NOTA: Este comando pode ser usado nos estados 1) *Modo de configuração* e 2) *Modo de leitura/comando*.

- h) **PID – Configuração de variáveis e parâmetros do controlador PID.** Este comando permite configurar o valor da posição desejada do disco, os parâmetros do controlador PID e a constante do filtro passa-baixo da ação derivativa. A implementação da funcionalidade deste comando será necessária para atingir os objetivos 6 e 7.

Definir parâmetros do controlador PID: $\langle char \rangle += \text{“PID } \beta \langle dig \rangle \beta \langle float \rangle +$

- i. $\langle dig \rangle = 0$ - Para definir a posição desejada para o disco.
- ii. $\langle dig \rangle = 1$ - Para definir a constante proporcional do controlador PID.
- iii. $\langle dig \rangle = 2$ - Para definir a constante integral do controlador PID.
- iv. $\langle dig \rangle = 3$ - Para definir a constante derivativa do controlador PID.
- v. $\langle dig \rangle = 4$ - Para definir a constante do filtro passa-baixo da ação derivativa do controlador PID.
- vi. $\langle dig \rangle = 5$ - Para solicitar os valores atuais dos parâmetros do controlador PID.
- vii. $\langle float \rangle$ - Valor definido para a posição desejada ou para os parâmetros do controlador PID.

NOTA: Este comando pode apenas ser usado no *estado 1) Modo configuração*. Por defeito, as constantes do controlador PID e a posição desejada devem estar configuradas com o valor 0.

O que enviar ao docente:

- Apresente o algoritmo da máquina de estados realizada, refletindo a lógica de funcionamento descrita no objetivo 0.

4.2 Implementação do cálculo de posição e velocidade do disco

A máquina elétrica MOT02043 já possui um *encoder* que gera **16 pulsos em quadratura por volta do motor**. Considerando a caixa redutora de **120:1**, cada volta do eixo corresponde a **1920 pulsos em quadratura** (Sensor A e Sensor B, em conjunto, cada um gera 960 pulsos). Com isto, deverão ser capazes de determinar a posição do disco com base no número de pulsos gerados pelo *encoder*, e a velocidade de rotação da carga com base no número de pulsos num dado período de tempo.

Se o microcontrolador contar os pulsos num intervalo de tempo conhecido, então a frequência dos pulsos (número de pulsos dividido pelo intervalo de tempo) dará uma medida da velocidade de rotação. A posição será a contagem dos pulsos a partir de uma referência que o utilizador considere como a posição zero do disco e que pode ser reinicializada.

O *encoder* SJ01 fornece dois sinais, desfasados entre eles 90°, o que torna possível detetar o movimento, e de igual forma, detetar o sentido de rotação. Na **Figura 5**, são apresentados dois casos de desfasamento dos sinais provenientes do *encoder* em função do sentido de rotação do eixo.

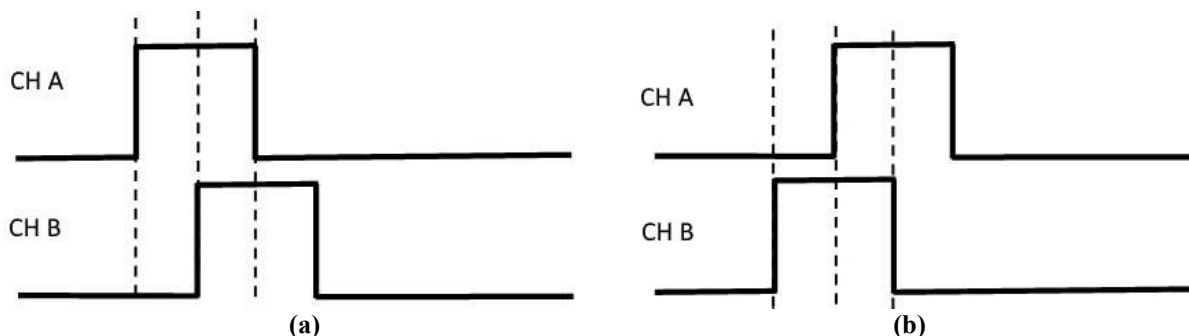


Figura 5 – Exemplificação dos pulsos gerados pelo *encoder*: **a)** a rotação em sentido horário provoca a subida da tensão no canal A antes do canal B; **b)** a rotação em sentido anti-horário provoca a subida da tensão no canal B antes do canal A.

Suponha-se que a saída do canal A do *encoder* esteja ligada a um pino de I/O do microcontrolador, que provoca uma interrupção à transição ascendente (ISR_P, que deve correr com máxima prioridade), e que a saída do canal B do *encoder* (v_B) esteja ligada a um pino de I/O a funcionar como *input*. Então, a rotina de interrupção começará por ler o valor do pino de I/O a que v_B está ligada. Se este for 0, o sentido de rotação será horário; caso contrário, será anti-horário. A rotina de interrupção incrementa (no sentido horário) ou decrementa (no sentido anti-horário) um contador de pulsos, realiza outras operações e retorna. Este contador de pulsos permite determinar a posição do disco.

Assim, o sinal de *feedback* do motor (**Sensor A**) deve ser ligado para provocar uma interrupção

cuja ISR corre com a máxima prioridade. A interrupção externa, originada pela transição de estado do pino de leitura do Sensor A, deverá invocar a função `ISR_P`, responsável por determinar o sentido de rotação (lendo o pino a que está ligada a saída do **sensor B**), incrementando (rotação no sentido horário) ou decrementando (rotação no sentido anti-horário) os contadores de pulsos (para a posição e a velocidade) e retornar. Este método deve apresentar um mecanismo de proteção caso o número de rotações exceda 10 voltas (número máximo de voltas admitido pelo contador de posição – nesse caso, o contador reinicia e avisa o utilizador).

Para a execução deste objetivo, deve-se iniciar pela implementação da medição de posição e velocidade. Para esta medição, é necessária outra ISR, que chamaremos de `ISR_T`, que exibirá os valores estabelecidos pela `ISR_P`. Para tal, deverá ser programado um temporizador que, com um período de amostragem de `HW` segundos (período de amostragem da posição/velocidade), gera uma interrupção que invoca a `ISR_T`. Esta adquire o valor do contador de pulsos da posição (*inc_pos*) e da velocidade (*inc_speed*) e coloca o contador de pulsos da velocidade em 0. O contador de velocidade, *inc_speed*, e o de posição, *inc_pos*, devem ser diferentes, pois o da posição apenas deve ser reinicializado quando solicitado pela respetiva instrução.

Para proceder ao cálculo da posição e da velocidade, será necessário definir uma interrupção do *timer* com base em um período de amostragem. Na STM32F767, os *timers* funcionam nas seguintes frequências (**Tabela 2**):

Tabela 2 - Frequências dos timers na TM32F767

Timer	1	2	3	4	5	6	7	8	9	10	11	12	13	14
APB2 (216MHz)	x							x	x	x	x			
APB1 (108MHz)		x	x	x	x	x	x					x	x	x

Para configurar de forma manual o período de amostragem, configurando a frequência com que o *timer* aciona a interrupção, pode usar-se a equação (1):

$$f_{IRQ} = \frac{f_{CLK}}{(PSC + 1)(ARR + 1)} \quad (1)$$

com:

- f_{CLK} – Frequência do *clock* do respetivo timer.
- PSC (*Prescaler*) – permite reduzir a frequência do *clock* do Timer
- ARR (*Auto-reload register*) – define o período do PWM

O tempo entre interrupções (T_{IRQ}), dado pela equação (2), em que ocorre a interrupção, é inversamente proporcional à frequência de interrupção (f_{IRQ}):

$$T_{IRQ} = \frac{1}{f_{IRQ}} \quad (2)$$

Objetivo 2 - Implementar a medida de posição e velocidade

Preparação de aula: devem preparar os algoritmos (esboços) a serem implementados (por exemplo, leitura do sentido de velocidade, conversão do número de pulsos em velocidade e em posição angular).

Depois de compreendido como devem realizar a leitura e o cálculo das medidas de posição e velocidade, devem agora proceder à sua implementação. Assim, será necessário disponibilizar os comandos funcionais de configuração do *período de amostragem* (HW), de *leitura* (R), do *tipo de leitura* (RT) e da *reinicialização da posição zero do disco* (RESPOS). Para os comandos de leitura de velocidade, o utilizador deve enviar o valor em SI, isto é, rad/s. Contudo, uma vez que o fabricante apresenta os valores nominais em rotações por minuto (RPM), uma unidade mais familiar para alguns utilizadores, este valor deve igualmente ser apresentado. Para o cálculo da medida da posição angular, é apenas necessário aceder ao contador do método da contagem de pulsos, enviando o valor em rad. Contudo, para maior facilidade de compreensão, podem enviar também em graus.

Metodologia de validação:

Para validar a algoritmia implementada da leitura de velocidade angular, deve seguir o seguinte procedimento:

- Desenvolver um pequeno programa que permita verificar se a interrupção associada ao período de amostragem está a ocorrer no instante correto. Para tal, deve ser gerada uma onda quadrada em que, a cada interrupção, o nível da saída alterna entre alto e baixo (ou vice-versa). Repetir este procedimento para períodos de amostragem de 5 ms e 10 ms. O sinal gerado deve ser observado no osciloscópio para confirmar o período obtido.
- Conectar a saída do gerador de sinais ao periférico de leitura do sensor A. O gerador de sinais deve ser configurado para cada um dos valores indicados na **Tabela 3** e ajustado aos limites de operação do periférico (ter cuidado com as amplitudes e o valor médio; só depois ligar o STM32). Por sua vez, o periférico do microcontrolador deve ser configurado para provocar uma interrupção no modo ascendente do pino.

Tabela 3: Tabela com a frequência aplicada ao sinal no gerador de sinais e leituras de velocidade obtidas na stm32

Frequência (Hz)	Velocidade (rpm)	ω (rad/s)
100		
500		
1000		

Para validar a algoritmia implementada da leitura de posição angular, deve seguir o seguinte procedimento:

- c) Conectar as saídas do *encoder* do motor ao periférico de leitura do sensor A e do sensor B. **Com os enrolamentos do motor em aberto (sem ponte completa)**, devem mexer manualmente o disco, alterando a sua posição. O movimento deve ser suave e não brusco, e o som das engrenagens não deve evidenciar qualquer anomalia mecânica. Realize os seguintes testes e preencha a **Tabela 4**:

Tabela 4: Cálculo da posição do disco

Rotação do disco	Leitura da posição (rad)	Leitura da posição (graus)
180°		
360°		
-180°		

- d) Validar a interface com o utilizador, configurando os registos necessários e analisando os valores obtidos;
- e) Reinicializar o contador da posição do disco via terminal, verificando o seu bom funcionamento.

Nota: Não se esquecer do limite máximo de contagem de voltas de 10 (para ambos os sentidos).

O que enviar ao docente:

- Envio os resultados obtidos no osciloscópio da onda quadrada gerada pela interrupção do período de amostragem para 5ms e 10ms.
- Um esboço do algoritmo desenvolvido para a leitura da posição e da velocidade;
- Enviar os resultados da **Tabela 3**;
- Enviar os resultados da **Tabela 4**, acompanhados de pequenos vídeos demonstrativos que mostrem a rotação do disco e a leitura da consola.

4.3 PWM

Um sinal PWM é um conjunto de pulsos retangulares com frequência fixa, f_{PWM} , logo, com período $T_c = 1/f_{PWM}$ também fixo. O valor de *duty cycle*, δ , é definido para cada período como a fração do período durante a qual o sinal de PWM está no estado alto. Este tempo simboliza-se por T_{DH} , e o tempo em que o sinal se encontra no estado baixo, por T_{DL} ; veja-se a equação (3). O *duty cycle*, δ , expressa o valor médio da tensão gerada como percentagem do valor máximo. Visto que o sinal de PWM tem frequência fixa, a soma de T_{DH} e T_{DL} é constante e igual ao período T_C (ver equação (4)). A frequência do sinal de PWM é igual ao inverso do período, conforme demonstrado na equação (5).

$$\delta = \frac{T_{DH}}{T_{DH} + T_{DL}} 100 \quad (3)$$

$$T_{DH} + T_{DL} = T_C \quad (4)$$

$$f_{PWM} = \frac{1}{T_C}$$

(5)

Na **Figura 6** estão representados 3 períodos de um sinal PWM. Repare-se que a soma dos tempos em estado alto (T_{DH}) e dos tempos em estado baixo (T_{DL}) é sempre constante e igual ao período do sinal (T_C), apesar de serem diferentes nos 3 períodos.

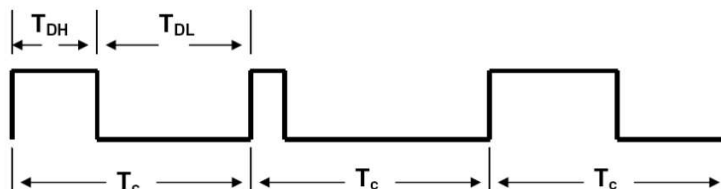


Figura 6 – Variação do sinal PWM ao longo de 3 períodos de comutação.

4.4 Conversor de eletrónica de potência em ponte completa

O conversor de eletrónica de potência CC-CA, vulgarmente designado por ponte H, permite gerar uma tensão bipolar $\{+E, -E\}$ ou unipolar $\{+E, -E \text{ e } 0\}$ a partir de um barramento CC de valor absoluto $\{E\}$, possibilitando a inversão do sentido de rotação de um motor CC. Conforme ilustrado na **Figura 7**, o acionamento baseia-se em três estados fundamentais: (*Off*) estado de 0 V na carga, obtido ligando simultaneamente os dois interruptores superiores ou os dois inferiores, permitindo travagem elétrica e evitando o funcionamento como gerador; (*Forward*) aplicação de $+E$ à carga, ligando dois interruptores em braços opostos, produzindo rotação num sentido; (*Reverse*) aplicação de $-E$, ligando o par complementar, invertendo o sentido de rotação. O funcionamento aprofundado deste conversor deve ser estudado previamente ao seu manuseio na unidade curricular correspondente.

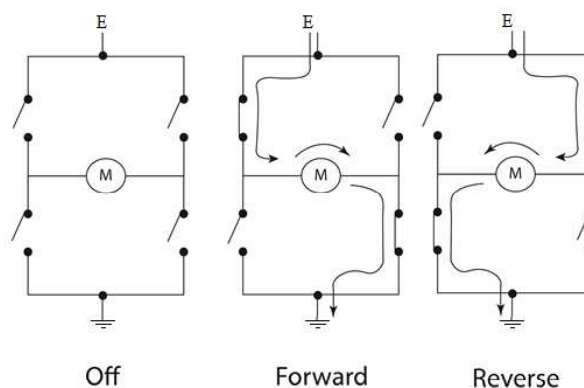


Figura 7 – Acionamento de um motor CC com uma ponte completa e respetivo sentido da corrente elétrica.

A tensão média aplicada à carga é controlada por PWM (*Pulse Width Modulation*). Na STM32, a geração de PWM é realizada por meio de *timers*. A frequência e o *duty cycle* do PWM são definidos pela configuração do *prescaler* e do contador do *timer*. A equação (6) apresenta a fórmula de cálculo da frequência:

$$f_{PWM} = \frac{f_{CLK}}{(PSC + 1)(ARR + 1)}$$

(6)

com:

- f_{CLK} – Frequência do *clock* do respetivo timer.
- *PSC (Prescaler)* – permite reduzir a frequência do *clock* do Timer
- *ARR (Auto-reload register)* – define o período do PWM

Relativamente ao *Duty Cycle* (equação (7)):

$$DutyCycle (\%) = \left(\frac{CCR}{ARR + 1} \right) \times 100 \quad (7)$$

com:

- *CCR (Capture/Compare Register)* - define o tempo em que o sinal PWM permanece em nível alto

Objetivo 3 – Implementação do comando para um canal PWM bipolar e acionamento do motor

Preparação de aula: devem ter realizado um esboço dos algoritmos a implementar. Devem fazer um estudo sobre como configurar o PWM no STM32.

Após compreender como gerar o PWM, será necessário implementá-lo. Neste trabalho, será usado um módulo da ponte completa, preferencialmente a BTS7960, acionado por um STM32. Alguns parâmetros são predefinidos, como a tensão CC da fonte de alimentação para fins de proteção (a tensão nominal do motor é de 6 V), o pino de PWM a utilizar (na BTS7960, dois pinos de PWM, um para cada sentido), o pino de ativação/desativação (ENABLE) e o pino de direção, utilizado na L298 (*forward/reverse* - FW/RV). A frequência do sinal PWM deve ser configurada em 1 kHz.

Nota: Os pinos podem variar dependendo do módulo da ponte completa adquirido, devendo ajustar a função lógica de *enable* e o controlo de direção ao respetivo módulo. A título de exemplo:

- Na BTS7960 existem dois pinos de PWM que controlam cada uma das direções de movimento. Há também dois pinos de ENABLE que devem estar ligados para que a ponte funcione. Deste modo, a movimentação do disco deve ser controlada por 2 sinais PWM, um para cada sentido.
- Na L298 há um pino de PWM e dois pinos que controlam a direção *forward* e *reverse*. A desativação simultânea dos dois pinos de direção torna a ponte completa *disable*. Depois, dependendo da escolha do pino, o PWM atua na direção desejada.
- O uso de outras pontes completas implica o estudo de seu modo de funcionamento.

Para garantir o cumprimento deste objetivo, será necessária a implementação da funcionalidade dos comandos do *duty cycle* (PWM). Devem complementar este comando com a capacidade de ajustar o *duty cycle* a partir do teclado, usando dois novos caracteres de controlo para que, de forma expedita, seja possível incrementar e decrementar o valor da tensão normalizada (sem necessidade de pressionar **ENTER**). Sugere-se, para isso, o carácter '/' para aumentar 5 unidades e o carácter '\' para reduzir 5 unidades. Verificar que a mudança do quadrante de operação seja suave e gradual (por exemplo, com uma mudança gradual do *duty cycle*: 10%; 5%; 0%; -5%; -10%). Evitar: 10%; 5%; 0%; -95%; -90%). Este comando apenas deve ser usado no estado 2) *Modo Leitura/comando manual*

Metodologia de validação:

- Comprovar o correto funcionamento do sinal PWM com o auxílio de um osciloscópio.
- Com o conversor de eletrónica de potência e a frequência de 1 kHz, manipular, **com cuidado**, o valor do *duty cycle* do sinal PWM de forma incremental até atingir 100 % (velocidade máxima de rotação do motor). Após atingir a velocidade máxima num dado sentido, reduzir progressivamente o *duty cycle* até à paragem do motor (*duty cycle* igual a 0) e observar/anotar o comportamento do sistema. Quando este estiver parado, efetuar o mesmo procedimento com o PWM negativo, decrementando-o com a tecla de controlo até ao valor de *duty cycle* de “-100 %”.
- Deverá ser observado e registado qual é o valor de PWM necessário para obter o movimento do veio (em ambos os sentidos).
- Registrar os valores de velocidade para cada 10 % de PWM (de -100 % a +100 %). Desenvolver um gráfico de linhas (**Excel**) com os dados adquiridos (*duty cycle* vs velocidade), apresentando o gráfico ao docente.

O que enviar ao docente:

- Enviar o comprovativo do sinal obtido no osciloscópio para frequência de 1 kHz e *duty cycle* de 0%, 25%, 50%, 75% e 100%
- Indicar qual *duty cycle* é necessário para o motor começar a mover, seja para rotação positiva ou negativa.
- Enviar o Excel com o comprovativo do comportamento do motor, com os dados de velocidade em função do *duty cycle* a 1 kHz, para uma sequência de valores de rotação positivos e negativos.

NOTA: É proibido a qualquer aluno retirar e guardar a maqueta nos armários. O docente deve entregar a maqueta a cada aluno, verificando previamente o seu correto funcionamento. No final da aula, o docente deve ainda confirmar que todas as maquetas estão devidamente organizadas.

Os alunos devem começar por verificar o correto acoplamento do eixo do motor ao veio mecânico do sistema. Devem também certificar-se de que o veio mecânico roda livremente, sem atrito significativo nem folga excessiva.

Durante os ensaios experimentais, devem verificar periodicamente a temperatura do motor e evitar sobrecarregar o sistema por períodos prolongados, uma vez que isso pode danificar o sistema de engrenagens.

A aplicação de tensão excessiva no motor ou a danificação das engrenagens implicará uma penalização de 40% em todos os objetivos do guia, ficando ainda os alunos responsáveis pela reparação adequada da maqueta.

5 Secção II – Módulos de controlo

5.1 Modelação do sistema

Neste sistema, a variável de comando u corresponde à tensão de alimentação a ser aplicada ao motor CC (o ganho do amplificador PWM é 1). As variáveis de estado do sistema são a velocidade angular ω e a posição angular θ do disco. Por conseguinte, basta um modelo do sistema mecânico e do motor CC com caixa de redução. Considerando o atrito apenas como linear e dinâmico, o modelo para o sistema mecânico é dado pelas equações (8) e (9):

$$J \frac{d\omega(t)}{dt} = m_f(t) + m_m(t) + m_d(t) \quad (8)$$

$$\begin{aligned} J \frac{d\omega(t)}{dt} &= -B\omega(t) + m_m(t) + m_d(t) \\ \frac{d\theta(t)}{dt} &= \omega(t) \end{aligned} \quad (9)$$

sendo m_f o binário de fricção, m_m o binário aplicado ao eixo pelo motor e pela caixa redutora, m_d qualquer binário de perturbação, J o momento de inércia do sistema mecânico e B o coeficiente de fricção linear.

Nas equações (8) e (9), deliberadamente ignoramos o efeito da fricção estática. As duas equações escalares podem ser combinadas numa equação vetorial de estado (equação de espaço de estados), como indicado na equação (10):

$$\begin{bmatrix} \frac{d\omega(t)}{dt} \\ \frac{d\theta(t)}{dt} \end{bmatrix} = \begin{bmatrix} -\frac{B}{J} & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \omega(t) \\ \theta(t) \end{bmatrix} + \begin{bmatrix} \frac{1}{J} & \frac{1}{J} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} m_m(t) \\ m_d(t) \end{bmatrix} \quad (10)$$

Em que:

- Vetor de estado: $\begin{bmatrix} \omega(t) \\ \theta(t) \end{bmatrix}$
- Vetor de entrada: $\begin{bmatrix} m_m(t) \\ m_d(t) \end{bmatrix}$
- Variável de atuação: m_m
- Variável de perturbação: m_d

O modelo do motor CC é dado por:

$$u_a(t) = Ri_a(t) + K_a\omega_a(t) \quad (11)$$

$$m_a(t) = K_a i_a(t) \quad (12)$$

Onde u_a é a tensão de alimentação, i_a a corrente de armadura, ω_a a velocidade angular do eixo do motor, m_a o binário produzido pelo motor no seu eixo, R a resistência de armadura e K_a a constante eletromecânica do motor.

Juntando as equações (11) e (12), o binário do motor é dado por:

$$m_a(t) = \frac{K_a}{R} (u_a(t) - K_a \omega_a(t)) \quad (13)$$

Assumindo que a caixa redutora tem uma razão de redução G e uma fricção insignificante, obtemos o seguinte:

$$m_m(t) = G m_a(t) \quad (14)$$

$$\omega_a(t) = G \omega(t) \quad (15)$$

Inserindo as equações (14) e (15) na equação do motor, obtemos:

$$m_m(t) = G m_a(t) = \frac{G K_a}{R} (u_a(t) - K_a G \omega(t)) \quad (16)$$

Definindo $K_m = G K_a$, obtemos o modelo para a combinação de motor e caixa redutora:

$$m_m(t) = \frac{K_m}{R} u_a(t) - \frac{K_m^2}{R} \omega(t) \quad (17)$$

A alimentação contínua da ponte completa pode resultar em um desvio em relação ao valor nominal. Isto implicará que o ganho do amplificador PWM possa desviar-se do valor nominal de 1. Para podermos lidar com esta situação, precisamos de a considerar explicitamente no modelo:

$$u_a(t) = K_w u(t) \quad (18)$$

A equação (18) indica que a tensão média de alimentação do motor u_a será igual ao valor de saída do controlador u , multiplicado pelo ganho K_w do amplificador PWM.

A equação do binário passa a ser:

$$m_m(t) = \frac{K_m K_w}{R} u(t) - \frac{K_m^2}{R} \omega(t) \quad (19)$$

Combinando a equação do modelo do motor, da caixa redutora e a equação de estado do sistema mecânico, obtém-se o modelo do espaço de tempo contínuo do sistema sobre o qual a regra de controlo funciona:

$$\begin{bmatrix} \frac{d\omega(t)}{dt} \\ \frac{d\theta(t)}{dt} \end{bmatrix} = \begin{bmatrix} -\frac{B}{J} - \frac{K_m^2}{RJ} & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \omega(t) \\ \theta(t) \end{bmatrix} + \begin{bmatrix} \frac{K_m K_w}{RJ} & \frac{1}{J} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u(t) \\ m_d(t) \end{bmatrix} \quad (20)$$

$$\begin{bmatrix} \omega(t) \\ \theta(t) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \omega(t) \\ \theta(t) \end{bmatrix} \quad (21)$$

A equação (20) é a equação de estado da forma $\frac{dx}{dt} = Ax + Bu$ que define as derivadas das variáveis de estado relativamente às variáveis de estado e à variável de entrada.

A equação (21) é a equação de medida ou de saída, na forma $y = Cx + Du$, que indica simplesmente que as variáveis do sistema que medimos são as variáveis de estado.

Nominalmente, e fruto de experiências realizadas e consulta de manuais, foram definidos os seguintes parâmetros para o sistema:

$$J = 0,02 \text{ kg/m}^2$$

$$B = 0,01 \text{ N/(m/s)}$$

$$K_m = 0,265 \text{ N/A}$$

$$R = 2,5 \text{ } \Omega$$

$$K_w = 1$$

5.2 Cálculo do valor dos parâmetros do sistema

Para tornar mais realista a simulação, aproximando aos valores dos parâmetros de cada uma das maquetas, é necessário determinar valores dos parâmetros, neste caso o J e B do modelo mecânico e K_m e R do motor e redutor. Na simulação, consideramos $K_w = 1$.

Para obter valores de parâmetros, devem ser realizados os seguintes procedimentos:

- Calcular com base nos primeiros princípios. Por exemplo, para obter o momento de inércia J , podemos calcular os momentos de inércia do disco e do eixo (aplicamos a expressão do momento de inércia de um cilindro com determinada massa e raio $J = \frac{1}{2}mr^2$) e somamos;
- Realizar experiências para deduzir os valores dos parâmetros a partir de medições, que podemos fazer em cada situação descrita por uma equação modelo;
- Verificar o *datasheet*.

As folhas de dados para motores de corrente contínua fornecem frequentemente, direta ou indiretamente, valores nominais para os parâmetros exigidos pelo modelo acima. No entanto, muitas vezes os valores nominais podem diferir dos valores reais; por isso, para maior precisão,

deve-se determinar experimentalmente os valores dos parâmetros. Para determinar o valor da constante $K_m = G.K_a$, é tido em conta as equações que descrevem o motor CC:

$$u_a(t) = Ri_a(t) + K_a\omega_a(t) \quad (22)$$

$$m_a(t) = K_a i_a(t) \quad (23)$$

e a caixa de velocidades:

$$m_m(t) = Gm_a(t) \quad (24)$$

$$\omega_a(t) = G\omega(t) \quad (25)$$

Substituímos ω_a e m_a nas equações do motor por suas respetivas expressões nas equações da caixa de velocidades. Obtém-se um modelo para o motor CC combinado e a caixa de velocidades como:

$$u_a(t) = Ri_a(t) + K_m\omega(t) \quad (26)$$

$$m_m(t) = K_m i_a(t) \quad (27)$$

O procedimento experimental para determinar K_m será:

- Medir R com um ohmímetro (em diferentes posições angulares da armadura para obter um valor médio);
- Aplicar uma tensão constante (não PWM) u_a ao motor;
- Aguardar que a velocidade se torne constante. Depois, medir:
 - A corrente i_a atual;
 - A velocidade ω .
- Introduzir os valores aplicados e medidos na equação:
 - $u_a(t) = Ri_a(t) + K_m\omega(t)$
- e calcular K_m .

Para calcular B :

- Para determinar o valor da constante B , observa-se que a primeira equação de estado escalar do sistema mecânico é a equação (26):

$$J \frac{d\omega(t)}{dt} = -B\omega(t) + m_m(t) \quad (26)$$

- Posteriormente, se for aplicado ao eixo um binário constante m_m (resultante do fornecimento do motor com uma tensão constante), a velocidade tornar-se-á constante, digamos, igual a ω_{ss} . Portanto:

$$0 = -B\omega_{ss} + m_m \leftrightarrow \quad (27)$$

$$B = \frac{m_m}{\omega_{ss}}$$

Para obter um binário constante, é necessário fornecer ao motor uma tensão constante. Sabendo o valor constante do K_m e medindo i_a (corrente) nesta situação, saberemos m_m ($m_m(t) = K_m i_a(t)$). A medição de ω_{ss} permite, então, calcular B pela equação (27).

Objetivo 4 – Análise e determinação dos parâmetros do sistema a controlar

Após a compreensão do método de determinação dos parâmetros do sistema, é necessário realizar, na maqueta, as experiências indicadas para determinar os parâmetros do motor e da fricção atribuídos ao grupo (devem ser sempre os mesmos). Os valores nominais dos parâmetros são:

$$J = 0,02 \text{ kg.m}^2$$

$$B = 0,01 \text{ N/(m/s)}$$

$$K_m = 0,265 \text{ N/A}$$

$$R = 2,5 \text{ } \Omega$$

$$K_w = 1$$

Estes têm de ser adaptados aos valores obtidos na maqueta atribuída a cada grupo. O momento de inércia do sistema mecânico (J) não vai ser calculado, devendo manter-se idêntico ao nominal: $0,02 \text{ Kg/m}^2$, e o ganho do PWM (K_w) vai ser considerado unitário. Todos os objetivos que se realizarão posteriormente devem ser sempre realizados com a mesma maqueta; caso contrário, podem surgir disparidades entre os resultados de simulação e os reais.

O que enviar ao docente:

- Explicar o procedimento experimental realizado, indicando os valores obtidos de R , i_a e ω , para o cálculo de K_m , e de m_m e ω_{ss} , para B .

5.3 Análise do modelo contínuo e importância dos polos do sistema

Depois de obtido o modelo do sistema a ser controlado, é preciso analisá-lo para identificar o seu comportamento em malha aberta, especialmente quanto à estabilidade. A informação fundamental sobre o comportamento de um modelo linear é fornecida pelos seus polos, ou valores próprios. Quando se determina uma função de transferência para o modelo, verifica-se que há valores de s que tornam o denominador da função de transferência igual a 0. Por esta razão, tais valores são denominados "polos".

Qual a informação fornecida pelos polos?

A importância dos polos do sistema provém das seguintes propriedades importantes:

- A resposta livre de um modelo linear (sistema) é dada por uma soma de n funções, onde n é o número de polos, ou seja, sabe-se que cada polo terá um termo correspondente na resposta livre.
- A resposta forçada de um modelo linear (sistema) é igual à transformação que o modelo (sistema) opera sobre a entrada, mais a soma das funções presentes na resposta livre.

Com a análise dos polos, podemos compreender a estabilidade do sistema: os polos devem estar todos no plano complexo esquerdo. A estabilidade estrita é imperativa para sistemas de controlo deste tipo.

Além disso, a distância dos polos ao eixo imaginário fornece informações sobre a rapidez com que as funções associadas (oscilantes ou não) se deterioram ou se ampliam. Se os polos forem um par complexo, a sua parte imaginária fornece-nos informação sobre a frequência de oscilação da função real associada.

Portanto, a primeira coisa a fazer após ter um modelo e valores assumidos para os parâmetros é analisar os valores numéricos dos polos.

O primeiro ficheiro de simulação a ser testado nos objetivos do trabalho é o *script* em **Matlab** “*lugar_raizes.m*”.

```

1  %Instruction required to clear variables and avoid errors
2  clear
3  %Nominal or undisturbed continuous time model
4  J=0.02;    %mechanical system moment of inertia
5  Bf=0.01;   %linear friction coefficient
6  Km=0.265;  %motor+gearbox electromechanical constant
7  R=2.5;     %armature resistance
8  Kw=1;      %PWM amplifier gain
9
10 %Continuous State space equation
11 A = [-Bf/J-Km^2/(R*J) 0 ; 1 0];
12 B = [Km*Kw/(R*J) 1/J; 0 0];
13 C = [1 0; 0 1];
14 D = [0 0; 0 0];
15 sys_c = ss(A,B,C,D)
16 %Nominal continuous model poles
17 p_c = pole(sys_c);
18 pzmap(p_c,[]);
19 disp("Nominal model poles in s:");
20 disp(p_c):

```

Figura 8 - Script Matlab: *lugar_raizes.m*

No *script Matlab* (**Figura 8**) estão definidos os valores dos parâmetros do sistema (são valores nominais e, naturalmente, diferentes dos que vão determinar na vossa maqueta), definido o valor do estado inicial para as variáveis do vetor de estado (ω e θ), e definida a equação do espaço de estados ($ss(A,B,C,D)$). A função *pole* permite determinar a localização dos polos do sistema. Para este sistema os polos são (**Figura 9**):

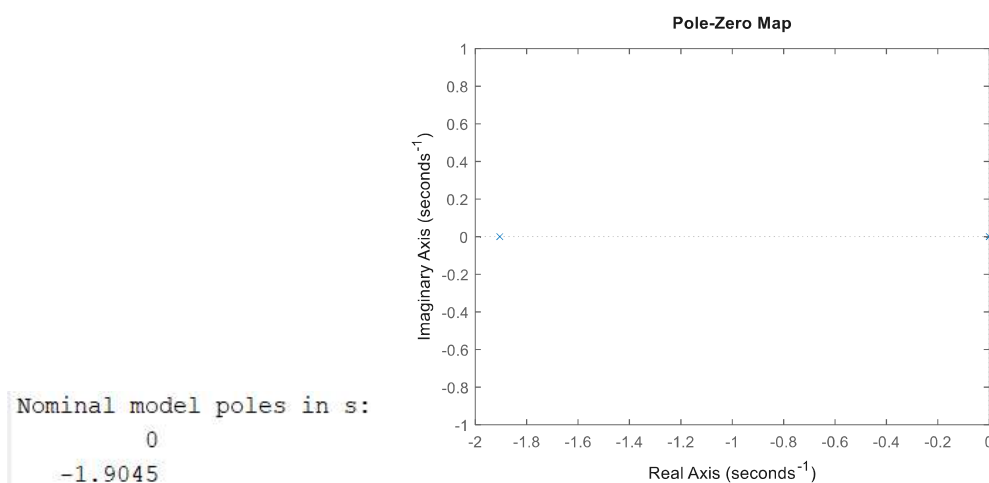


Figura 9 - Polos do sistema e representação no lugar de raízes.

5.4 Cálculo do período de amostragem

Na conceção de um controlador digital, a primeira escolha a fazer é definir o valor do período de amostragem h . Embora não exista um critério ótimo para a escolha, normalmente define-se que o período de amostragem deve ser tal que o controlador consiga detetar, de forma significativa, na resposta do sistema, a presença da função de mudança mais rápida associada a um polo. Esta função deve ser associada ao polo de maior módulo (correspondente à maior frequência de corte na resposta de frequência do sistema). No sistema a ser considerado, o polo com maior módulo é o polo a $-1,9$. É possível observar que a resposta livre está associada a um exponencial decrescente, com constante de tempo $T = 1/1,9 = 0,53$.

Assumindo como regra que, para detetar significativamente esta exponencial decrescente, recolher 4 a 10 amostras durante um intervalo de tempo igual a T . Então o período de amostragem h terá de satisfazer:

$$\frac{T}{10} \leq h \leq \frac{T}{4} \quad (28)$$

Assim, para este caso deve usar-se:

$$h = \frac{T}{8} = \frac{0,53}{8} = 0,065789 \approx 0,065 \quad (29)$$

5.5 Modelo discreto do sistema

Nominalmente, quando controlamos um sistema de tempo contínuo por meio de um dispositivo digital com período de amostragem h , estaremos a forçar a entrada $u(t)$ a ser uma sequência de impulsos. Isto é assim porque $u(t)$ só pode mudar de valor nos instantes de amostragem kh e permanece constante até ao instante de amostragem seguinte:

$$kh \leq t \leq kh + h \rightarrow u(t) = u(kh) \quad (30)$$

Quando um sistema de tempo contínuo com modelo linear:

$$\begin{aligned} \frac{dx(t)}{dt} &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) + Du(t) \end{aligned} \quad (31)$$

É forçada, como indicado em cima, a equação de estado temporal discreta que relaciona o seu estado no instante $kh+h$ com o estado e a entrada no instante kh é dada por:

$$x(kh + h) = \Phi(h)x(kh) + \Gamma(h)u(kh) \quad (32)$$

com:

$$\Phi(h) = e^{Ah}$$

$$\Gamma(h) = \int_0^h e^{A\tau} d\tau B$$

(33)

Em **Matlab**, a conversão entre o modelo de espaço de estados contínuo e o modelo discreto pode ser realizada usando o comando:

```
sys_d=c2d(sys_c,h);
```

Assim, a função *c2d* realiza a conversão do modelo de espaço de estados contínuo, definido pela variável *sys_c*, com o período de amostragem *h*, e cria o espaço de estados discreto *sys_d*. Na Figura 11, é possível visualizar um excerto do código do *script* que implementa a conversão do modelo de espaço de estados contínuo para o discreto. Para realizar esta conversão, é também necessário ter definido o valor do período de amostragem (*h*).

```
%Sampling period
h=0.065; %Approximately T/8
disp("Sampling period:")
disp(h);

%Continuous State space equation
A = [-Bf/J-Km^2/(R*J) 0 ;1 0];
B = [Km*Kw/(R*J) 1/J; 0 0];
C = [1 0; 0 1];
D = [0 0; 0 0];
sys_c = ss(A,B,C,D);

%2. ----- Calculation of discrete models and controller parameters
sys_d=c2d(sys_c,h); % Convert space states model from continuous to discrete
Fi=sys_d.A; % Fi = A
Gama=sys_d.B; % Gama = B
C_d = sys_d.C;
D_d = sys_d.D;
```

Figura 10 – Exemplo de um *script* que converte o modelo contínuo do sistema em discreto

5.6 Análise do sistema/processo em malha fechada

Concluída a análise do comportamento do sistema em malha aberta, o passo seguinte é definir um controlador que nos garanta o controlo da variável controlada, isto é, que permita que, no nosso sistema, a posição angular do disco tenda para a posição angular definida como referência. Um dos controladores que possibilitam isto é o PID.

5.6.1 Controlador PID Convencional

O controlador PID convencional na forma paralela é descrito pela expressão:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d e(t)}{dt}$$

(34)

Em que $u(t)$ é a variável de comando, $e(t)$ é a variável de erro, K_p é o ganho proporcional, K_i é o ganho integral e K_d é o ganho derivativo. A variável de erro é igual à variável de referência menos a variável controlada, $e(t) = y_r(t) - y(t)$. No caso da CP2, y será sempre entendida como a posição angular do disco. As três parcelas no membro direito da equação são usualmente chamadas ação proporcional, ação integral e ação derivativa, visto que cada uma é,

respetivamente, proporcional à integral do erro, proporcional ao erro e proporcional à derivada do erro.

No domínio de Laplace tem-se:

$$U(s) = \left[K_p + K_i \frac{1}{s} + K_d s \right] E(s) \quad (35)$$

Consoante se ponha a 0 o ganho integral, o ganho derivativo ou ambos, obtêm-se as variantes do PID listadas na tabela a seguir.

Tabela 5 - Tabela das variantes dos controladores PID

Variante	K_p	K_i	K_d
Proporcional (P)	> 0	$= 0$	$= 0$
Proporcional Integral (PI)	> 0	> 0	$= 0$
Proporcional Derivativa (PD)	> 0	$= 0$	> 0
Proporcional Integral Derivativa (PID)	> 0	> 0	> 0

Os critérios de escolha de cada uma das variantes são os seguintes.

- i) Controlador P: quando apenas com o ganho proporcional se consegue uma precisão adequada para o erro em regime permanente e boa resposta transitória;
- ii) Controlador PI: como em i) relativamente à resposta transitória, mas é requerido ou conveniente ter uma melhor precisão no erro em regime permanente (de preferência nulo);
- iii) Controlador PD: como em i) relativamente à precisão em regime permanente, mas é requerido ou conveniente melhorar a resposta transitória;
- iv) Controlador PID: a utilizar quando as características do problema de controlo requerem ação integral e derivativa para atingir boa precisão em regime permanente e boa resposta transitória.

5.6.2 Aproximação Digital do Controlador PID Convencional

Para aproximar digitalmente, no tempo, a equação (36), com período de amostragem h , pode-se usar a seguinte expressão em tempo discreto.

$$u(kh) = K_p e(kh) + K_i \sum_{i=1}^k \frac{[e(ih) + e((i-1)h)]}{2} h + K_d \frac{e(kh) - e((k-1)h)}{h} \quad (36)$$

Na equação (36), h representa o período de amostragem e k é o índice dos instantes de amostragem kh . O erro no instante kh é representado por $e(kh) = y_r(kh) - y(kh)$.

5.6.3 Simulação do controlo PID em Matlab

Para implementar o controlo do sistema em malha fechada em **Matlab**, é necessário criar um modelo em **Simulink**, utilizando o bloco PID como módulo de controlo. Foi criado um *script*

básico denominado *pid_script.m* (ver **Figura 11**), que permite a criação do modelo no espaço de estados a partir dos parâmetros definidos no objetivo 4.

```

1      %Instruction required to clear variables and avoid errors
2      clear
3      %Nominal or undisturbed continuous time model
4      J=0.02;      %mechanical system moment of inertia
5      Bf=0.01;     %linear friction coefficient
6      Km=0.265;    %motor+gearbox electromechanical constant
7      R=2.5;       %armature resistance
8      Kw=1;        %PWM amplifier gain
9
10     %Sampling period
11     h=0.065;      %Approximately T/8
12     disp("Sampling period:")
13     disp(h);
14
15     %Continuous State space equation
16     A = [-Bf/J-Km^2/(R*J) 0 ;1 0];
17     B = [Km*Kw/(R*J) 1/J; 0 0];
18     C = [1 0; 0 1];
19     D = [0 0; 0 0];
20     sys_c = ss(A,B,C,D);
21
22     %2. ----- Calculation of discrete models and controller parameters
23     sys_d=c2d(sys_c,h); % Convert space states model from continuous to discrete
24     Fi=sys_d.A;        % Fi = A
25     Gama=sys_d.B;      % Gama = B
26     C_d = sys_d.C;
27     D_d = sys_d.D;
    
```

Figura 11 - Script *pid_script.m*

Este deve ser executado na primeira utilização, para que os seus dados possam ser utilizados no bloco *discrete state-space* do **Simulink**. O modelo **Simulink** criado denomina-se *pid_simulink.xls* e é apresentado na **Figura 12**.

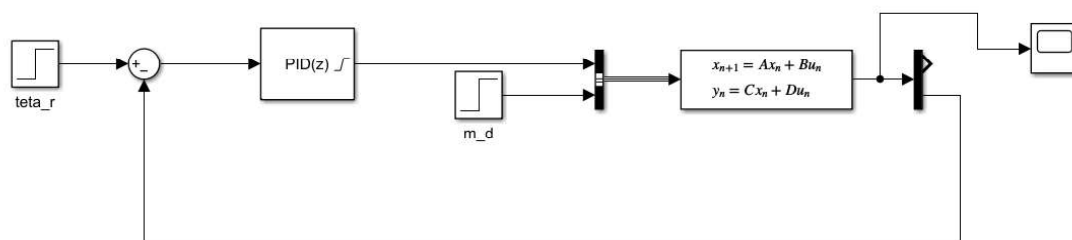


Figura 12 - Esquemático em **Simulink** do sistema em malha fechada.

No modelo **Simulink**, pode-se especificar a posição desejada no bloco *step*: *teta_r* (no parâmetro *Final value* do bloco – na **Figura 14**, foi aplicada uma posição desejada de 2π), o valor do binário de perturbação, no bloco *step*: *m_d* e os parâmetros do controlador PID, no bloco *PID controller*.

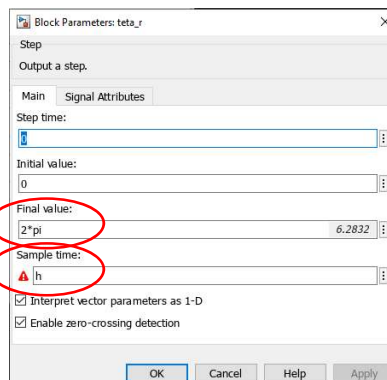


Figura 13 – Imagem do bloco *step* da posição desejada, com valor de posição desejada igual a 2π (final value), período de amostragem igual a h (sample time) e os demais elementos com valor 0.

No bloco do PID (ver **Figura 14**), é possível indicar o tipo de controlador (P, PI ou PID) e, dependendo do tipo seleccionado, definir os valores de ganho proporcional, integral e derivativo. O valor do *sample time* permanece igual ao h definido por vocês no *script* em **Matlab**.

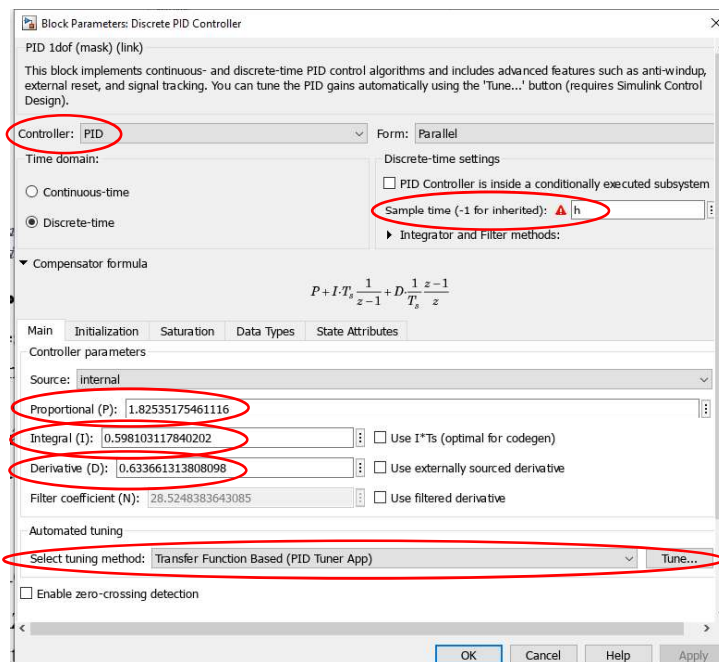


Figura 14 - Bloco PID do Simulink

O objetivo desta simulação é verificar o comportamento do sistema, para determinados parâmetros do controlador PID. Estes devem garantir o menor erro em regime permanente possível, com ou sem perturbação, e uma resposta que minimize o *overshoot*. O processo de seleção dos parâmetros adequados para atender aos requisitos pretendidos chama-se sintonização. Uma forma expedita de sintonizar os controladores PID é usar o *PID Tuner App* do **Matlab**.

5.6.4 PID Tuner App

Na Figura 14, é possível visualizar como pode ser seleccionada a sintonização dos parâmetros do controlador PID, usando o *PID tuner app* e fazendo *Tune*. Ao seleccionar esse botão, surge uma nova janela (ver Figura 15), na qual é possível definir alguns parâmetros da resposta, como o tempo de resposta mais rápido ou mais lento (*response time*) e a resposta transitória mais ou menos agressiva (*transient behavior*). Ao alterar esses parâmetros, a resposta é atualizada no gráfico e, ao fazer um *update block*, os parâmetros são automaticamente atualizados no bloco do controlador. Os parâmetros alterados e o tipo de resposta dependem de seleccionarmos, no bloco PID, um controlador P, PI ou PID.

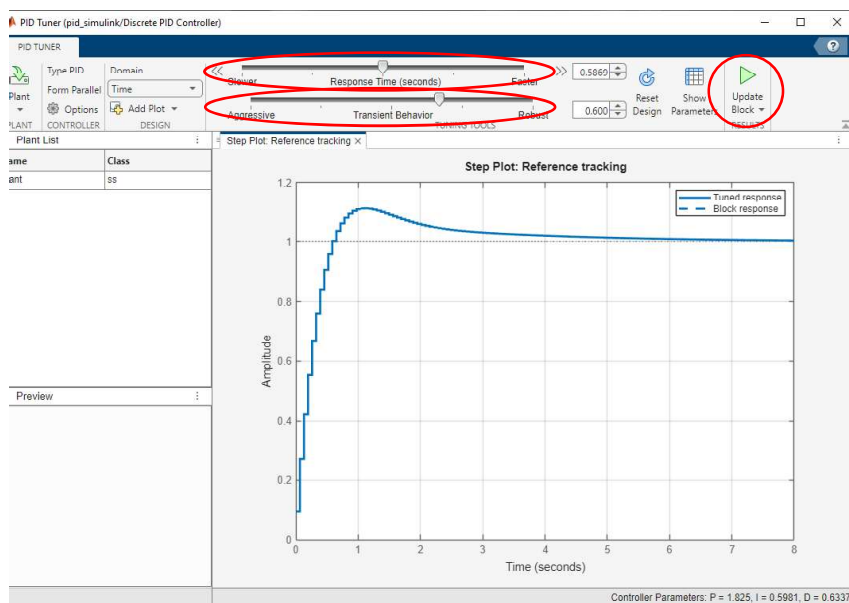


Figura 15 - PID tuner app.

Objetivo 5 – Cálculo em simulação, do período de amostragem e dos parâmetros do controlador PID

Com os parâmetros determinados no objetivo 4, é agora possível obter o modelo de espaço de estados do sistema em simulação. Como o objetivo da CP é modelizar e controlar o sistema em modo discreto, será necessário, inicialmente, determinar o período de amostragem. Isso pode ser obtido a partir da determinação dos polos do sistema. Assim, em simulação, será necessário criar o modelo no espaço de estados do sistema e, depois, determinar a localização de seus polos (para já, tudo em contínuo). Para isso, será usado o *script lugar_raizes.m*. Neste *script*, é necessário ajustar os parâmetros do sistema aos valores obtidos no objetivo 4. Depois, ao executar o *script*, obtém-se o modelo de espaço de estados em contínuo e o diagrama com a localização dos polos do sistema. A partir da localização dos polos, é possível calcular o período de amostragem (h) para a discretização do sistema.

Depois de determinado o período de amostragem, como referido na secção 5.5, é fácil converter o modelo contínuo em discreto em **Matlab** (ver o *script pid_script.m*). Com o modelo de espaço de estados do sistema discreto já obtido, é possível modelar o sistema em malha fechada e determinar os parâmetros do controlador PID em ambiente de simulação. Para tal, será utilizado o *PID Tuner app* do **Matlab** (ver secção 5.6.4) para o cálculo dos parâmetros do controlador nos modos P, PI e PID. Não se esquecer da necessidade de utilizar os vossos parâmetros da

maqueta no modelo de espaço de estados e na definição do período de amostragem. Para isso, na primeira utilização, devem utilizar o *script* *pid_script.m*, para criar o modelo de espaço de estados discreto a partir dos vossos parâmetros. Esse modelo de espaço de estados será usado pelo **Simulink** no bloco que o define (ver secção 5.6.3). O valor do período de amostragem (h) também é utilizado em vários blocos do modelo **Simulink**. Usando o ficheiro **Simulink**: *pid_simulink.xls*, podem realizar o teste do modelo em malha fechada, fornecendo valores à posição desejada (bloco *step_teta_r*), ao valor da perturbação (bloco *step_m_d*) e ao ajuste dos parâmetros do controlador PID ($PID(Z)$).

Metodologia de validação:

- i. Fazer o lugar de raízes e calcular o valor do período de amostragem a utilizar.
- ii. Determinar o parâmetro de ganho proporcional do modelo e verificar a resposta do sistema na posição desejada $\pi/2$.
- iii. Adicione uma perturbação externa e verifique a resposta do sistema (aplicar no bloco *step_m_d* do **Simulink** o valor 0,02)? O controlador foi imune à perturbação externa?
- iv. Determinar os parâmetros do controlador PI e verificar a resposta do sistema para uma posição desejada de $\pi/2$?
- v. Verificar novamente a resposta do sistema a uma perturbação externa (aplicar no bloco *step_m_d* do **Simulink** o valor 0,02). O sistema foi imune à perturbação?
- vi. Determinar os parâmetros do controlador PID e verificar a resposta do sistema para uma posição desejada de $\pi/2$? Quais as vantagens e desvantagens do uso do PID?
- vii. Verificar novamente a resposta do sistema a uma perturbação externa (aplicar no bloco *step_m_d* do **Simulink** o valor 0,02)? Compare o comportamento em relação aos controladores P e PI?

Deste estudo devem resultar os parâmetros do controlador PID. Os valores dos parâmetros podem ser posteriormente aplicados ao controlador PID, por meio do comando (PID) definido no objetivo 1.

O que enviar ao docente:

- Mostrar o lugar de raízes do sistema, explicando qual o procedimento realizado para determinar o período de amostragem;
- Os alunos devem preparar um gráfico **Excel** com os resultados obtidos para o sistema para o controlo P, PI e PID (indicando os valores dos parâmetros), para uma resposta sem e com perturbação;
- Verificar em que casos o sistema foi imune à perturbação;
- Explicar as vantagens e desvantagens de cada controlador.

5.7 Aproximação Digital do Controlador PID

Como indicado anteriormente, o controlador PID convencional pode ser aproximado digitalmente no tempo por meio da equação 36. Para efeitos de cálculo do valor $u(kh)$, pode ser escrita de forma mais adequada. Uma simplificação óbvia é destacar as constantes na aproximação da integral e da derivada (equação (38)), e reescrever o somatório dos erros de forma recursiva (equação (39)), definindo a variável $sum_e(k)$, e substituir na equação (40):

$$u(kh) = K_p e(kh) + \frac{K_i h}{2} \sum_{i=1}^k [e(ih) + e(i-1)h] + \frac{K_d}{h} [e(kh) - e(kh-1)h] \quad (37)$$

$$K_{p_h} = K_p \quad K_{i_h} = \frac{K_i h}{2} \quad K_{d_h} = \frac{K_d}{h} \quad (38)$$

$$sum_{e(k)} = \sum_{i=1}^k [e(i) + e(i-1)] = sum_{e(k-1)} + e(k) + e(k-1) \quad (39)$$

$$u(k) = K_{p_h} e(k) + K_{i_h} sum_e(k) + K_{d_h} [e(k) - e(k-1)] \quad (40)$$

Com as considerações acima, uma forma possível de escrever a rotina de interrupção (ISR), em pseudocódigo, do controlador PID convencional, pode ser:

```
ISR_PID()
    y=input(y)
    e=yr-y
    sum_e=sum_e+e+e_ant
    u=Kp_h*e+Ki_h*sum_e+Kd_h*(e-e_ant)
    e_ant=e
    output(u)
    return from interrupt
```

A primeira instrução obtém o valor de $y(k)$, a segunda calcula $e(k)$, a terceira atualiza o somatório de erros usando o valor de $e(k-1)$ guardado em e_ant , a quarta calcula o valor de $u(k)$, também usando o valor de $e(k-1)$ guardado em e_ant para a ação derivativa, a quinta

guarda o valor de $e(k)$ em e_ant para utilização na invocação da rotina no instante $k + 1$ e a sexta faz a saída do valor do comando, para memória, um gerador de PWM. A variável sum_e , o somatório dos erros, deve ser declarada como *float*.

Para a operação do controlador PID, o utilizador especifica quatro parâmetros no programa de interface: h , K_p , K_i e K_d .

5.7.1 O controlador D na variável medida e PI no erro

Em aplicações industriais, é comum utilizar a seguinte variante da família de regras PID. Esta variante é aproximadamente³ derivativa na variável medida, proporcional na variável de erro e aproximadamente integral⁴ na variável de erro. De um ponto de vista prático, a ação derivativa do erro pode resultar em elevada variância e picos na variável de comando, geralmente denominados “*derivative kick*”. Por outro lado, a derivativa na variável medida aumenta a sensibilidade às mudanças na variável de referência (*set point*), o que é mais adequado para aplicações práticas.

Assim, em aplicações industriais, a variável de comando é calculada da seguinte forma, em cada invocação k da ISR:

$$u(k) = -K_d \frac{y(k) - y(k-1)}{h} + K_p(y_r(k) - y(k)) + K_i \sum_{i=0}^{k-1} (y_r(i) - y(i)) h \quad (41)$$

Na expressão acima, K_d , K_p e K_i são denominados ganhos derivativo, proporcional e integral, respetivamente. O período de amostragem é h . A variável controlada é y e a variável de referência é y_r .

Para efeitos de cálculo do valor $u(k)$ a equação (41) pode ser escrita de forma mais adequada. A simplificação consiste em calcular o erro só uma vez (equação (42)), pôr em evidência as constantes na aproximação à integral e à derivada (equação (43)), e calcular o somatório de erros de forma recursiva (equação (44)):

$$e(k) = y_r(k) - y(k) \quad (42)$$

$$K_{p_h} = K_p \quad K_{i_h} = K_i h \quad K_{d_h} = \frac{K_d}{h} \quad (43)$$

$$sum_e(k) = \sum_{i=0}^{k-1} e(i) = sum_e(k-1) + e(k-1) \quad (44)$$

Obtemos então a forma de calcular a equação (45) como

$$u(k) = -K_{d_h}[y(k) - y(k-1)] + K_{p_h}e(k) + K_{i_h}sum_e(k) \quad (45)$$

³ O “aproximadamente” resulta de se usar uma aproximação causal discreta à derivada.

⁴ O “aproximadamente” resulta de se usar uma aproximação discreta ao integral.

Note-se que $K_{i_h} \neq K_i$ e $K_{d_h} \neq K_d$. Os valores dos parâmetros K_{p_h} , K_{i_h} e K_{d_h} devem ser definidos pelo programa de interface com o utilizador, pela equação 45, porque não faz sentido, sendo constantes, estarem a ser continuamente recalculados na rotina de interrupção.

Valores máximos e mínimos de K_{p_h} , K_{i_h} e K_{d_h} , dependem da gama de valores aceites para K_p , K_i e K_d . Sempre que houver uma mudança nestes valores ou no período de amostragem h , os valores de K_{p_h} , K_{i_h} e K_{d_h} devem ser recalculados pelo programa de interface, fazendo *reset* do somatório dos erros e da variável anterior de saída.

Com as considerações acima, o pseudocódigo para a implementação do controlador D na variável medida e PI no erro pode ser definido da seguinte forma:

```
ISR_PID()
    y=input(y)
    e=y-r-y
    sum_e=sum_e+e_ant
    u=Kp_h*e+Ki_h*sum_e-Kd_h*(y-y_ant)
    e_ant=e
    y_ant=y
    output(u)
    return from interrupt
```

5.7.2 Programação da regra de controlo com filtro passa-baixo na ação derivativa

A utilização de um filtro passa-baixo na ação derivativa pode ajudar a reduzir o ruído e os erros de medida. A equação da ação derivativa pode escrever-se:

$$u_d(k) = \frac{K_d}{h} [y(k) - y(k-1)] \quad (46)$$

Aplicando um filtro passa-baixo de primeira ordem, com a seguinte equação

$$y[n] = ay[n-1] + (1-a)x[n-1] \quad (47)$$

O filtro na equação (47) é digital, similar a um filtro analógico RC. A resposta do filtro a um degrau de tensão corresponde à amostragem da resposta de um circuito RC, cuja constante de tempo é dada por $T = RC$.

O valor de a varia no intervalo $]0, 1[$. Com valores de a próximos de 0, o valor de $y[n]$ acompanha facilmente o de $x[n]$. Com valores próximos da unidade, as variações no valor de $x[n]$ refletem-se lentamente no de $y[n]$. A **Figura 16** mostra a resposta do filtro a um degrau de tensão na entrada do ADC, para valores de $a = 0,1, 0,3$ e $0,9$.

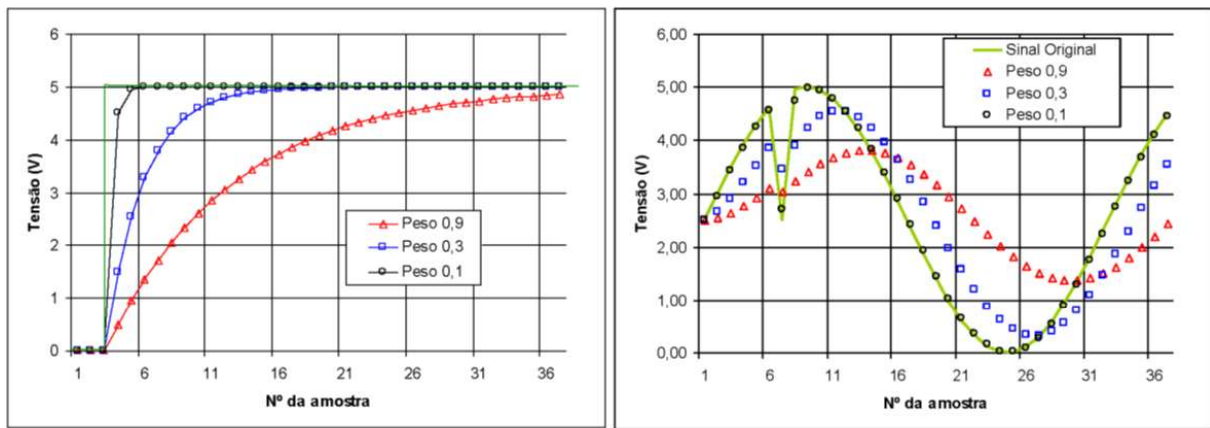


Figura 16 - Resposta do filtro passa-baixo a um degrau de tensão na entrada do ADC, com valores de $a = 0,1, 0,3$ e $0,9$.

Podemos reescrever a equação da ação PID como:

$$u_d(k) = au_d(k-1) + \frac{K_d(1-a)}{h} [y(k) - y(k-1)] \quad (48)$$

Podemos definir genericamente a expressão de K_{d_h} como sendo:

$$K_{d_h} = \frac{K_d(1-a)}{h} \quad (49)$$

Esta equação reduz-se à anterior se a for 0.

Para a operação do controlador PID com a utilização de um filtro passa-baixo na ação derivativa, o utilizador especifica, na interface, cinco parâmetros: os anteriores h , K_p , K_i , K_d e o parâmetro a . A ação derivativa sem filtro passa-baixo é obtida com $a = 0$, o que implica $K_{d_h} = K_d / h$.

O pseudocódigo para o algoritmo PID com filtro passa-baixo na ação derivativa passa a ser definido por (modificações a amarelo):

```
ISR_PID()
    y=input(y)
    e=y-r-y
    sum_e=sum_e+e_ant
    u_d=Kd_h*(y-y_ant)+a*u_d_ant
    u=Kp_h*e+Ki_h*sum_e-u_d
    e_ant=e
    y_ant=y
    u_d_ant=u_d
    output(u)
    return from interrupt
```

5.7.3 Limitação do valor do comando aos valores de saturação e paragem na atualização dos erros

Se, como resultado do cálculo, o valor do comando for superior ao valor de saturação superior, ou inferior ao valor de saturação inferior é necessário limitar o mesmo aos valores de saturação. Além disso, a saturação do atuador pode dar origem ao “disparo” dos valores do somatório dos erros, sum_e , levando a oscilações prejudiciais na variável controlada

```
ISR_PID()
  y=input(y)
  e=y-r-y
  sum_e_bkp=sum_e //in case u becomes saturated
  sum_e=sum_e+e_ant
  u_d=Kd_h*(y-y_ant)+a*u_d_ant
  u=Kp_h*e+Ki_h*sum_e-u_d
  e_ant=e
  y_ant=y
  u_d_ant=u_d
  if u>U_sat_a then //u above upper saturation
    u=U_sat_a
    sum_e=sum_e_bkp //sum of errors frozen,
    //back to last value
  else if u<U_sat_b then //u below lower saturation
    u=U_sat_b
    sum_e=sum_e_bkp //sum of errors frozen,
    //back to last value
  output(u) //unsaturated u remains unchanged
  return from interrupt
```

Objetivo 6 - Implementação do algoritmo do controlador PID e validação da algoritmia implementada

Preparação de aula: o pseudocódigo para a implementação do controlador PID é fornecido. É, no entanto, necessário perceber o que cada variável representa e verificar se há variáveis que devem ser reinicializadas ao início de uma nova experiência de controlo (por exemplo, os somatórios).

Implementação

Este objetivo visa a implementação do controlador digital PID no microcontrolador para cálculo da variável de comando $u(k)$. Este código será utilizado no *estado 3) Modo automático*. A variável controlada será a posição angular da carga rotativa. Os valores da variável de referência são fornecidos pela interface, considerando o sistema de unidades SI, conforme implementado na Parte 1. Para obter os valores da variável medida, é necessário utilizar as funções de cálculo da posição angular do motor a partir do sinal em quadratura do *encoder* (utilizando o algoritmo do Objetivo 2). Na programação do controlo, pode utilizar os parâmetros do controlador PID

obtidos em simulação (Objetivo 5) e ajustá-los ao protótipo real, conforme necessário. Por último, deve integrar a variável de comando à atuação em PWM (utilizando o algoritmo do Objetivo 3).

O algoritmo digital PID é implementado como uma ISR, designada ISR_PID, invocada com o período h (definido na interface pelo comando HW).

Para além da programação do algoritmo PID digital, pretende-se também implementar funcionalidades comuns em controladores industriais, como o filtro na ação derivativa, a deteção de saturação e a limitação da ação integral.

```
ISR_PID()
    y=input(y)
    e=y-r-y
    sum_e_bkp=sum_e      //in case u becomes saturated
    sum_e=sum_e+e_ant
    u_d=Kd_h*(y-y_ant)+a*u_d_ant
    u=Kp_h*e+Ki_h*sum_e-u_d
    e_ant=e
    y_ant=y
    u_d_ant=u_d
    if u>U_sat_a then    //u above upper saturation
        u=U_sat_a
        sum_e=sum_e_bkp    //sum of errors frozen,
                           //back to last value
    else if u<U_sat_b then //u below lower saturation
        u=U_sat_b
        sum_e=sum_e_bkp    //sum of errors frozen,
                           //back to last value
    output(u)           //unsaturated u remains unchanged
    return from interrupt
```

Note que, devido às características do sistema que está a controlar, U_sat_a (valor de saturação superior de atuação) e U_sat_b (valor de saturação inferior de atuação) correspondem a 6 V e -6 V, respetivamente.

Metodologia de validação:

Nesta fase, pretende-se validar a algoritmia implementada no controlador PID digital, por meio dos seguintes testes. Primeiro, é necessário testar a programação do cálculo da variável de comando $u(k)$. Segundo, comparar a resposta do sistema mecânico, com e sem o filtro, na ação derivativa.

Para o teste do algoritmo do controlador PID, é importante verificar se o cálculo do valor de comando a partir do valor da variável medida está a ser realizado adequadamente. Para isso, devem colocar o controlador em funcionamento, calcular o valor de comando $u(k)$ sem aplicá-lo à saída do atuador e usar individualmente cada uma das ações P, I ou D (testando uma de cada vez). Deve colocar o controlador numa situação em que o sinal de erro seja conhecido.

Deve fazê-lo, programando na memória do microcontrolador os valores de um sinal de tensão em onda quadrada ou triangular (usar os valores dos gráficos das Figura 17, Figura 18, Figura 19 e Figura 20) e considerando a variável de referência constante. Testa-se a ISR_PID fazendo-a ler os valores de y de memória. Nesta condição, o sinal de erro $e(k)$ fica bem definido e o sinal de comando $u(k)$ também, consoante seja seleccionada a ação P, I ou D. Os valores de entrada da onda quadrada ou triangular e os resultados obtidos para $u(k)$ devem ser enviados para o terminal, e depois reproduzidos no Excel. Em seguida, apresentam-se exemplos de sinais consistentes.

Coloque-se em memória uma onda retangular de 0 a 2 V, garantindo que tenha valor médio de 1 V. Ao mesmo tempo, ajusta-se o valor da variável de referência para corresponder ao valor médio de 1 V. Então, a variável de erro $e(k)$ terá valores entre -1 V e 1 V, pois $e(k) = 1 - y(k)$ ⁵. Como $e(k)$ tem uma evolução conhecida, pode-se determinar como deve ser a evolução de $u(k)$ para os valores de K_p , K_i e K_d estabelecidos.

Por exemplo, se o período de amostragem for de 10 ms e a onda retangular tiver uma frequência de 5 Hz, e se forem utilizados $K_p = 1,5$, $K_i = 0$ e $K_d = 0$ (**apenas ação proporcional**), as evoluções de $y(k)$ e $u(k)$ deverão ter o aspeto representado na **Figura 17**. Note-se que as figuras a seguir não mostram o erro, e sim a entrada.

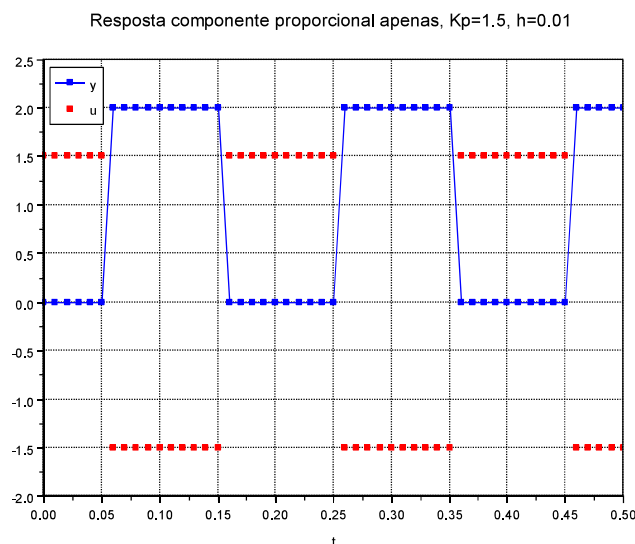


Figura 17 - Resposta da componente proporcional a uma entrada que provoca um erro em onda retangular. Referência igual a 1.

Se fizermos $K_p = 0$, $K_i = 20$ e $K_d = 0$ (**apenas ação integral**) para a mesma entrada, as evoluções de $y(k)$ e $u(k)$ deverão ter um aspeto semelhante ao representado na **Figura 18**, dependendo do instante preciso em que o algoritmo começa a integrar o erro.

⁵ $1 - 0 = 1$; $1 - 2 = -1$

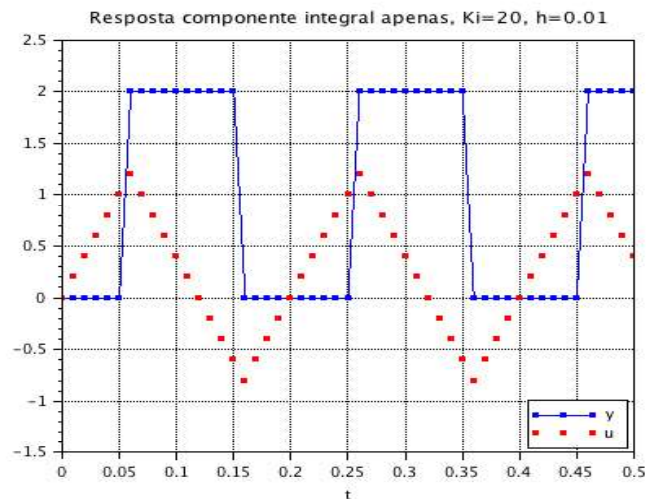


Figura 18 - Resposta da componente integral a uma entrada que provoca um erro em onda retangular. As pendentes da onda triangular dependem do valor de K_i . O valor médio e o desfasamento da mesma dependem do instante em que o algoritmo começa a integrar a variável u .

Para testar a componente derivativa, deve-se armazenar em memória uma onda triangular entre 0 e 2 V. Como a onda triangular tem derivada constante, provoca, na saída, uma onda retangular. Se fizermos $K_p = 0$, $K_i = 0$ e $K_d = 0,05$ (**apenas a ação derivativa**), para uma entrada triangular (0 a 2 V) em y , as evoluções de $y(k)$ e $u(k)$ deverão ter o aspeto representado na **Figura 19**. Note-se que a ação derivativa deve ser afetada por um sinal negativo.

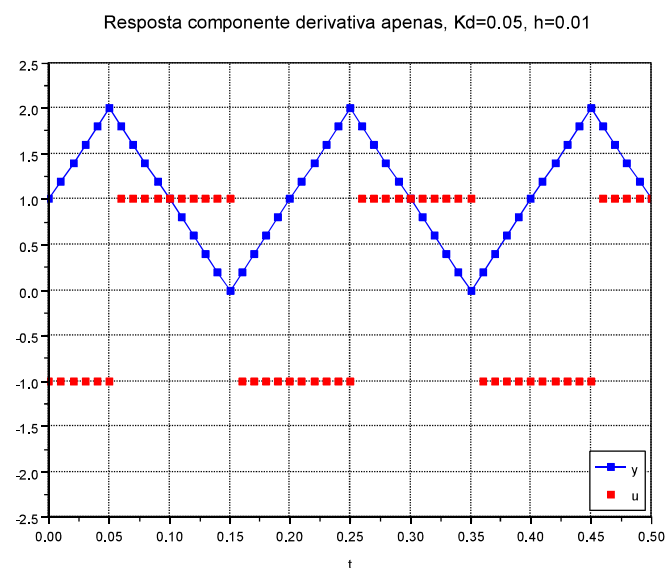


Figura 19 - Resposta da componente derivativa a uma entrada em onda triangular. A derivada é uma onda retangular. Note que a ação derivativa tem sinal oposto ao da derivada.

Para o teste da ação derivativa com filtro passa-baixo, supomos a mesma onda triangular de entrada e $K_p = 0$, $K_i = 0$, $K_d = 0,05$ e $a = 1/3$, conforme a **Figura 20**.

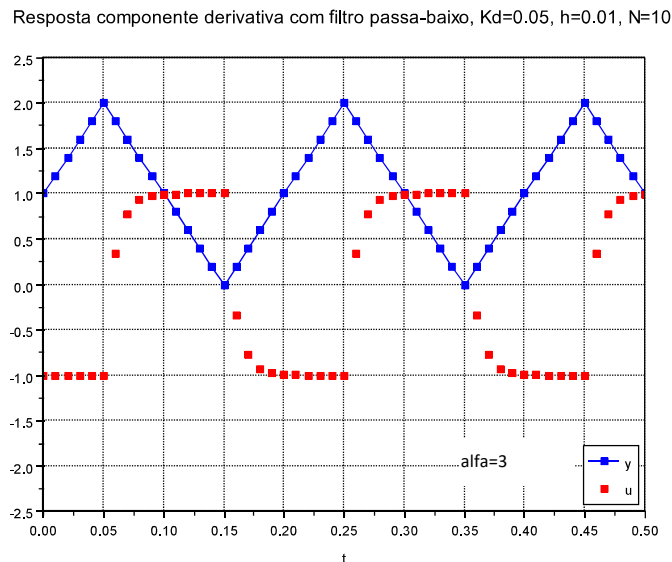


Figura 20 - Resposta da componente derivativa com filtro passa-baixo a uma entrada que provoca um erro em onda triangular. A derivada é uma onda retangular com uma aproximação exponencial.

No final, deve-se verificar a ação do filtro na ação derivativa, alterando o valor de a para 0, 2/3 e 1. Compare os resultados com os obtidos na **Figura 20**.

O que enviar ao docente:

- Mostrar a validação do algoritmo de controlo, mostrando, em primeiro lugar, a resposta do controlador à entrada de um sinal retangular, com apenas a ação proporcional.
- Depois mostrar a resposta do controlador aplicando apenas a ação integral, para o mesmo sinal de entrada;
- Posteriormente, mostrar o comportamento do controlador aplicando um sinal triangular e a ação apenas derivativa, sem filtragem;
- Mostrar o comportamento do controlador para a ação derivativa e com filtragem;
- Por fim, realize uma análise crítica da alteração dos valores na ação derivativa de 0, 2/3 e 1.

Objetivo 7 - Avaliação experimental do controlador

Neste objetivo, pretende-se avaliar o comportamento experimental do sistema de controlo digital implementado no protótipo real apresentado na **Figura 1**. Pretende-se também investigar as vantagens e desvantagens do uso de controladores P, PI e PID no controlo do disco da maquete.

Para avaliar o sistema de controlo, é necessário realizar o teste da posição do disco da maquete para diversos valores de posição definidos pelo utilizador. Assim, deve-se ajustar os parâmetros dos diversos tipos de controlador para garantir o funcionamento o mais adequado possível para o disco. Deve-se também avaliar o comportamento do controlador perante perturbações externas (por exemplo, perturbações manualmente aplicadas pelo utilizador), para os diversos tipos de controlador.

O que mostrar ao docente:

- Mostrar o comportamento do disco da maquete à aplicação de diferentes posições desejadas para o disco, com o controlo proporcional;
- Aplicar uma perturbação ao sistema e verificar qual o comportamento do disco;
- Realizar o mesmo procedimento para o controlador PI, com e sem aplicação de perturbação externa;
- Por último, realizar o mesmo procedimento para o controlador PID;
- Realize uma análise crítica da alteração dos parâmetros de controlo e do tipo de controlador.